

REVIEW

Open Access



Source code vulnerability detection based on deep learning: a review

Huading Su^{1,2}, Zhen Xu¹, Yan Zhang^{1,2*}  and Qian Tan¹

Abstract

Source code vulnerability detection is a crucial aspect of software security development, and the current use of Large Language Models (LLMs) accelerates not only software development but also the generation and propagation of code vulnerabilities. Traditional code vulnerability detection techniques have limited detection efficiency and accuracy. Deep learning techniques have recently gained distinct advantages in multidimensional feature extraction and large-scale data processing, and their application in code vulnerability detection is evolving from simple classification to multimodal approaches. This paper primarily systematizes and summarises deep learning-based source code vulnerability detection, as well as analyzes and anticipates current challenges and future research directions in this area. The distinction between this review and the preceding reviews: This study investigates the literature of the last four years; Not only does it contain datasets, but also includes model-related research and an analysis of multiple different application scenarios. It's more current and comprehensive than most previous reviews of this type.

Keywords Software security, Source code, Vulnerability detection, Deep learning, Model interpretability

Introduction

Software technology has become the core driving force of the information society. As the basic elements of software composition, code inevitably contains various vulnerabilities. The impact of these security vulnerabilities has permeated key areas including technical infrastructure, economic systems, social governance, and national security. Global supply chain vulnerabilities (e.g., Log4j (Doll et al. 2025)) show a significant “butterfly effect”. Notably, foreign Advanced Persistent Threat (APT) organizations frequently exploit zero-day vulnerabilities to conduct network attacks. The U.S. Cybersecurity and Infrastructure Security Agency (CISA) report indicates that most energy system attacks in 2023 originated from unpatched

vulnerabilities (CISA 2025). More alarmingly, generative artificial intelligence (AI) technology has exacerbated code vulnerability issues: while AI coding tools improve efficiency by generating code, they simultaneously introduce new types of compound vulnerabilities, and attackers can leverage LLM prompts to mass-produce attack scripts. China National Vulnerability Database (CNVD) statistics for 2015–2024 (see Fig. 1) demonstrate the trend of code vulnerability changes, underscoring the severity of the current cybersecurity landscape.

Code vulnerabilities are design flaws or implementation errors in a program that can cause it to deviate from its intended goal, and may be exploited by attackers to undermine system security. Its core performance: flaws in the program's logic or security policy can be triggered by specific inputs, resulting in information leakage, privilege elevation, system crash, and other security risks.

The evolution of code vulnerability detection techniques has progressed through three distinct phases: rule-based approaches, machine learning methods, and deep learning paradigms. Early rule-based systems (e.g., Fortify

*Correspondence:

Yan Zhang
zhangyan@iie.ac.cn

¹ Institute of Information Engineering, Chinese Academy of Sciences, No. 19, Shucun Road, Haidian District, Beijing 100085, China

² School of Cyber Security, University of Chinese Academy of Sciences, Beijing, China

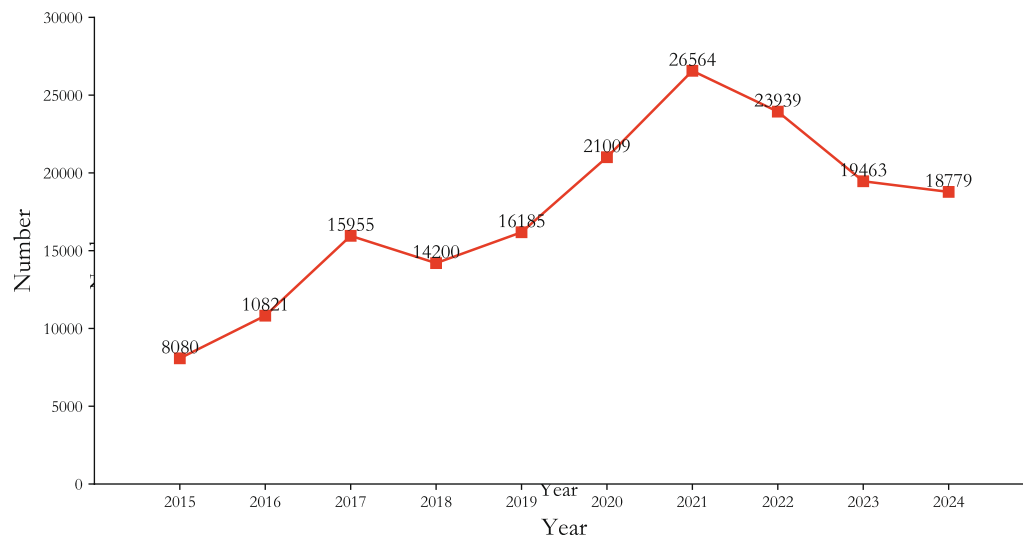


Fig. 1 Trends in code vulnerabilities based on CNVD data

(Chess And McGraw 2004), FindBugs (Hovemeyer And Pugh 2004), and Checkmarx (2025)) employed manually defined regular expressions and syntax tree-matching rules. While effective for identifying known vulnerability patterns, these methods exhibited significant limitations including high False Positive Rate (FPR), poor rule generalization, and substantial maintenance costs. The machine learning phase introduced vectorized code characterization capabilities through techniques such as association rule mining (Neuhaus et al. 2007), ensemble learning (Shin And Williams 2008), and decision tree-based feature analysis (Yamaguchi et al. 2012). However, these approaches remained constrained by feature engineering complexity and insufficient semantic understanding. Current deep learning techniques (Feng et al. 2020; Zhou et al. 2021; Ahmad et al. 2021) have enhanced feature extraction depth, and emerging LLM technologies demonstrate cross-task potential through Agent collaboration with Retrieval Augmentation Generation (RAG) knowledge bases (Dozono et al. 2024). Nevertheless, persistent challenges include training data dependency, limited model generalization capacity, and high computational resource requirements.

This paper focuses on a summary overview of source code vulnerability detection techniques in the deep learning phase, aiming to answer the following research questions: RQ1: What are the current status and limitations of benchmark datasets for deep learning-based source code vulnerability detection RQ2: What are the key technical challenges in deep learning-based source code vulnerability detection RQ3: What are the programming language distributions, vulnerability patterns, and

corresponding detection challenges across diverse application scenarios

Existing research in deep learning-based code vulnerability detection has made some progress. Among these studies, the literature (Shi-Kun 2023; Su et al. 2024) systematically organises the relevant technical frameworks from a deep learning theoretical perspective, whereas the literature (Senanayake et al. 2023; Chu et al. 2023; Kumar et al. 2024; Kaniewski et al. 2024; Anand et al. 2024) focuses on detection methodologies for specific application scenarios. However, these works exhibit two critical limitations: (1) they do not incorporate the most recent advancements in this field; (2) they lack cross-scenario comparative analyses. In comparison, the main contributions of this paper are as follows:

- Firstly, this paper presents a relatively comprehensive literature review, which is newer in time than most existing related surveys. The research for the first two RQs consists of the following: 1) benchmark datasets for code vulnerability detection, 2) deep learning-based source code vulnerability detection techniques, 3) interpretability analysis of deep learning detection models, and 4) evaluation metrics and their practical applications.
- Secondly, for RQ3 we analyse and summarise the programming languages, vulnerability characteristics, and detection challenges associated with code vulnerability detection across different scenarios.
- Finally, we analyze and summarize both the current challenges in deep learning-based source code vulnerability detection and potential future research directions.

Research questions

The research questions addressed in this paper are as follows:

- **RQ1: What are the current status and limitations of benchmark datasets for deep learning-based source code vulnerability detection?** This research question addresses two primary objectives: 1) systematic classification and statistical analysis of mainstream benchmark datasets for code vulnerability detection, and 2) comprehensive examination of inherent limitations in these datasets' construction and application. The evaluation framework considers multiple critical dimensions, including data sources, class balance, and annotation granularity, to evaluate dataset utility.
- **RQ2: What are the key technical challenges in deep learning-based source code vulnerability detection?** This research question aims to systematically investigate the core technical bottlenecks in applying deep learning to code vulnerability detection. The study will adopt a three-phase analytical framework: 1) examining the evolutionary trajectory and

performance boundaries of mainstream approaches; 2) analyzing how model interpretability limitations affect detection reliability, and 3) summarizing the existing mainstream evaluation indicators.

- **RQ3: What are the programming language distributions, vulnerability patterns, and corresponding code vulnerability detection challenges across diverse application scenarios?** This research focuses on several different scenarios. The study systematically investigates: 1) The distribution patterns of programming languages and vulnerability types across these scenarios, and 2) Scenario-specific technological challenges in vulnerability detection.

This paper presents a systematic literature review on the application of deep learning in source code defect detection. For each research question, we employed different keyword combinations for literature retrieval, with specific search terms and results detailed in Table 1. Following the research methodology of Zhang et al. (2011), we identified primary studies answering research questions (RQs) through a rigorous literature

Table 1 The search results of the research questions

RQs	Keywords	IEEE Electronic Library	ACM Digital Library	Google Scholar	arXiv	Wanfang Data	CNKI	Total Count
RQ1	("vulnerability" ∨ "defect" ∨ "bug") ∧ ("code detection") ∧ ("deep learning") ∧ ("dataset")	116	84	914	109	73	1,099	2,395
RQ2	("vulnerability" ∨ "defect" ∨ "bug") ∧ ("code detection") ∧ ("deep learning")	158	91	1,120	194	279	1,241	3,083
RQ3	("vulnerability" ∨ "defect" ∨ "bug") ∧ ("code detection") ∧ ("deep learning") ∧ (("mobile application") ∨ ("smart contract") ∨ ("code generation"))	6/ 15/ 11	29/ 33/ 47	49/ 111/ 160	1/ 9/ 83	1/ 46/ 20	140/ 99/ 294	226/ 313/ 615

The symbol ∨ denotes the logical OR operator, while ∧ represents the logical AND operator. For RQ3, which contains several different application scenarios, the document retrieval counts are presented in an "X/Y/Z" format. In this notation, "X" indicates the count of relevant documents for the mobile application scenario, "Y" represents the count for the smart contract scenario and "Z" represents the count for the code generation scenario

screening process. The screening comprised four phases: Initially, we queried major academic databases including IEEE Xplore, ACM Digital Library, Google Scholar, arXiv, CNKI, and Wanfang Data using predefined keywords, searching approximately 6,600 publications. In the second phase, we filtered documents based on full-text availability and duplication, reducing the corpus to over 5,700 papers. During the third phase, manual verification of publication venues, titles, and abstracts further refined the selection to 430 eligible studies. Finally, we conducted a full-text review to exclude irrelevant literature while incorporating seminal works published outside the 2021-2025 journal/conference scope. Exclusion criteria included superficial mentions of deep learning without substantive methodology application, and studies unrelated to source code vulnerability detection. Ultimately, more than 140 relevant articles were selected for comprehensive analysis.

Organization of the review

The overall research framework is illustrated in Figure 2, comprising three RQs. The rest of this article is organized as follows: Section [Background](#) introduces the background knowledge. Section [Datasets of source code vulnerability detection](#) addresses RQ1 by examining datasets for source code vulnerability detection. Section [Deep](#)

[learning-based source code vulnerability detection models](#) corresponds to RQ2, focusing on deep learning models for this task. Section [The study of application scenarios](#) relates to RQ3, investigating multiple application scenarios. Section [Challenges and future work](#) summarizes the challenges and future research directions of deep learning-based source code vulnerability detection. Section [Conclusion](#) concludes the article.

Background

This section begins by formally defining source code vulnerability detection, followed by a concise introduction to deep learning-based approaches for source code vulnerability detection.

Vulnerability detection formulation

Vulnerability Detection. Source code vulnerability detection is formulated as a classification task: given a source code sample represented by feature vector $X_i \in R^d$ (where d is the feature dimension), the model predicts its category label $Y_i \in \gamma$ ($1 \leq i \leq N$, N is the total number of samples). The category set γ contains n mutually exclusive classes ($n \geq 2$), where one specific class is defined as “non-vulnerability” (denoted as c_0), and the remaining $n - 1$ classes correspond to different types of vulnerabilities (denoted as $c_1, c_2, \dots, c_{n-1}$).

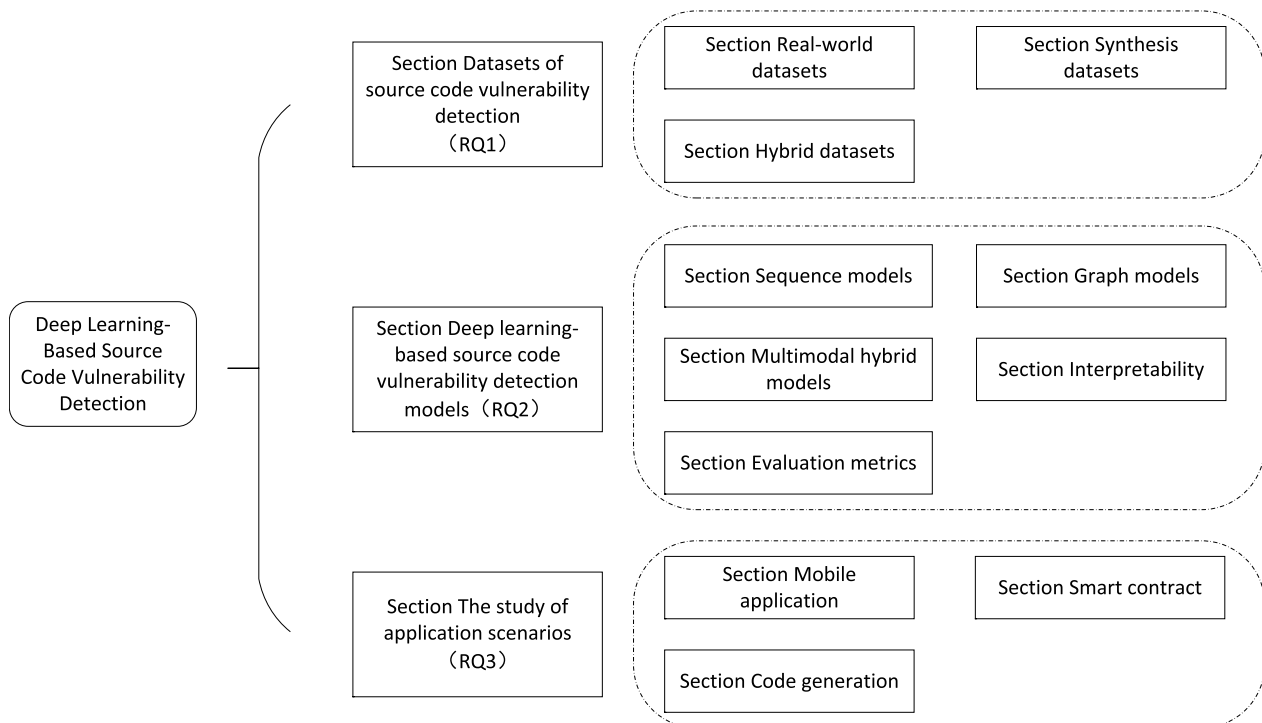


Fig. 2 Structure of this systematic literature review article

Example 1

Binary classification: When $n = 2$, the category set is $\gamma = \{c_0, c_1\}$, where: c_0 represents “non-vulnerability” (the code sample has no vulnerability); c_1 represents “vulnerability” (the code sample contains at least one vulnerability, regardless of specific type). For a given source code input X_i , the model predicts:

- $Y_i = c_1$ if the sample is vulnerable;
- $Y_i = c_0$ if the sample is non-vulnerable.

In practical applications, labels are often encoded as integers for simplicity: $Y_i = 1$ (vulnerable) and $Y_i = 0$ (non-vulnerable).

Example 2

Multi-class classification: When $n > 2$, the category set is $\gamma = \{c_0, c_1, c_2, \dots, c_{n-1}\}$, where: c_0 represents “non-vulnerability”; $c_k (1 \leq k \leq n-1)$ represents specific vulnerability types. For a given source code input X_i , the model predicts a unique label from γ :

- $Y_i = c_k (1 \leq k \leq n-1)$ if the sample contains the k -th type of vulnerability;
- $Y_i = c_0$ if the sample is non-vulnerable.

Integer encoding is commonly used: $Y_i = k (1 \leq k \leq n-1)$ for vulnerability types, and $Y_i = 0$ for non-vulnerability.

Deep learning-based approaches

Traditional code vulnerability detection techniques are primarily categorized into static and dynamic analysis based on whether the program needs to be executed (Wang et al. 2025). Static analysis detects vulnerabilities by parsing source code but suffers from high false-positive rates (Li et al. 2018); while dynamic analysis offers improved accuracy, it is constrained by test case quality and Code Coverage (CC), and often fails due to incomplete source code acquisition. Hybrid approaches combining static and dynamic methods can mitigate false positives and negatives but struggle with low detection efficiency (Votipka et al. 2018).

Recently, deep learning has significantly enhanced detection generalization by automatically learning vulnerability patterns in code, providing more efficient and automated solutions. This paper focuses on source code vulnerability detection using deep learning, which can be further divided into sequence-based models, graph-based models, and hybrid multimodal models, depending on the data format processed by the model (Wu And Zou 2022). Figure 3 illustrates the workflow of deep learning-based source code vulnerability detection, comprising three main components: input, model processing, and output. The input can be source code or processed into code property graphs (CPGs). The model processing

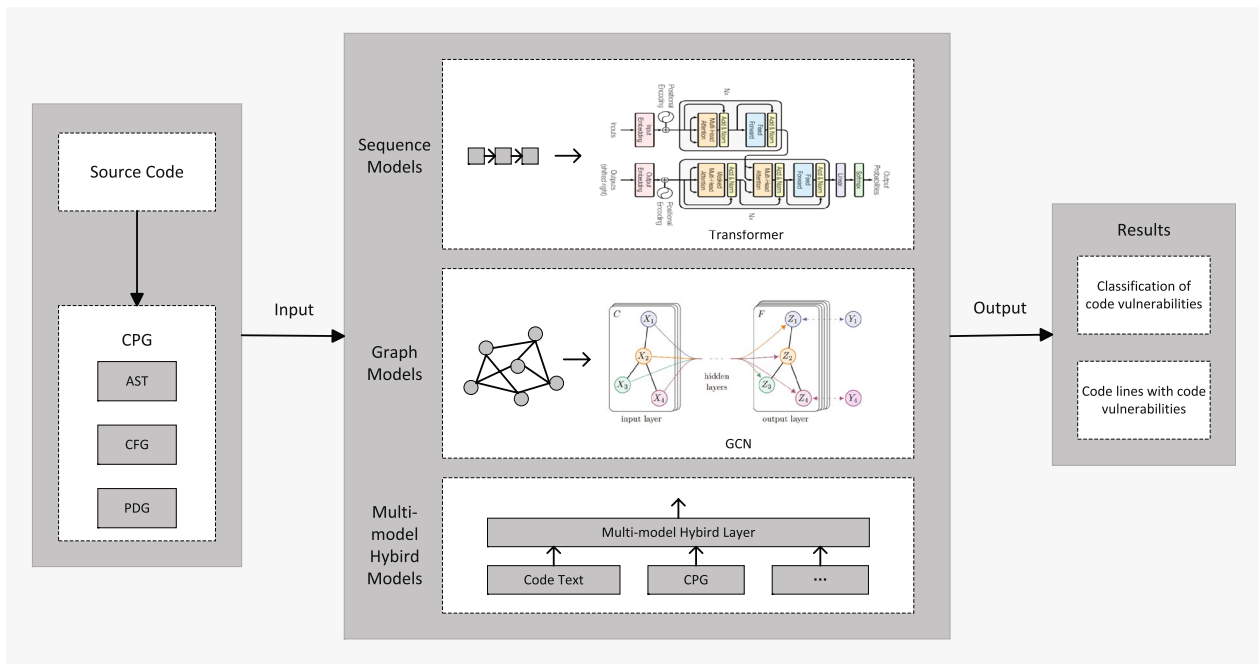


Fig. 3 Deep learning-based source code vulnerability detection pipeline

stage employs various deep learning architectures to extract and analyze features from source code and CPGs. The output may include vulnerability classification results or identification of vulnerable code lines.

Sequence models

Recurrent Neural Networks (RNNs) are classical models for processing sequential data. Their core mechanism captures temporal dependencies by iteratively updating hidden states (i.e., memory). Traditional RNNs suffer from the vanishing gradient problem, leading to the development of improved variants such as Long Short-Term Memory (LSTM) networks and Gated Recurrent Units (GRUs). In the domain of code vulnerability detection, RNN-based approaches typically follow this workflow, as follows(Zou et al. 2019): first, code is segmented into token sequences; subsequently, enhanced models such as Bidirectional LSTM (BLSTM) networks are employed for feature extraction and vulnerability prediction. This methodology effectively captures contextual relationships and long-range dependencies in code.

The Transformer(Vaswani et al. 2017), proposed by the Google team in 2017, is a sequential model whose breakthrough lies in replacing traditional RNN recurrent architectures with self-attention mechanisms. By computing the correlation weights between sequence elements in parallel, the Transformer efficiently addresses both long-range dependencies and training inefficiency. This architecture comprises an encoder and decoder. The encoder extracts contextual features through multi-head self-attention layers and feedforward networks, while the decoder employs masked self-attention mechanisms for sequence generation. The introduction of positional encoding compensates for the absence of temporal information. Currently prominent large language models (LLMs) predominantly adopt the Transformer architecture, including GPT(Brown et al. 2020), LLaMA(Touvron et al. 2023), and Deepseek(DeepSeek-AI et al. 2025). Through training on vast datasets, LLMs learn syntactic rules, semantic associations, and reasoning abilities. They not only generate and comprehend natural language but also integrate knowledge bases to answer factual questions.

Graph models

Graph Neural Networks (GNNs)(Wu et al. 2019) update node representations by aggregating neighbor information through a message-passing mechanism, effectively capturing the topological structure of graph data. They find extensive applications in social network analysis and molecular structure prediction. Among these, Graph Convolutional Networks (GCNs)(Wang et al. 2020) achieve information aggregation by normalizing the adjacency matrix and applying linear transformations, employing either spectral or spatial domain

approaches. Graph Attention Networks (GATs)(Velickovic et al. 2017), building upon GCNs, introduce attention mechanisms to dynamically assign weights to neighboring nodes. Multi-head attention further enhances model expressiveness, rendering GATs particularly suitable for heterogeneous graphs or scenarios requiring dynamic focus on critical neighbors. Collectively, these approaches advance the efficient modeling and analysis of graph-structured data. Typically, employing GNNs to process code-related data necessitates first converting code text into a code representation graph. The following outlines the code graph structure:

- **Abstract Syntax Tree (AST):** The AST is a structured, tree-like representation of source code, describing the syntactic structure of program code while omitting specific syntactic details (e.g., semicolons, whitespace).
- **Control Flow Graph (CFG):** The CFG is a directed graph composed of basic blocks (unbranched sequences of consecutive instructions) and edges (control transfers), illustrating program execution paths. Nodes represent basic blocks, while edges denote conditional jumps or sequential execution.
- **Data Flow Graph (DFG):** The DFG traces the flow of data through CFGs, typically encompassing variable accesses and modifications.
- **Program Dependency Graph (PDG):** The PDG is a directed graph whose nodes correspond to those in the CFG, excluding entry/exit nodes. The edges connecting PDG nodes represent the control and data dependencies between them. Consequently, it encompasses both the Data Dependency Graph (DDG) and the Control Dependency Graph (CDG).
- **Code Property Graph (CPG):** The CPG integrates the property characteristics of AST, CFG, and PDG to form a complete graph structure, combining the advantages of these three graph types.

Multimodal hybrid models

The definition of multimodal typically encompasses two forms: narrow sense (Bayouddh 2024) and broad sense (Zhou And Verma 2022a). The narrow sense specifically refers to native data types directly corresponding to human sensory modalities, primarily including text, images, audio, and video. The broad sense extends to different representations of the same object, not limited to biological sensory modalities. The multimodal discussed in this paper is primarily considered from a broad perspective. A multimodal hybrid model refers to a model capable of simultaneously processing information from multiple modalities, enhancing perception and decision-making capabilities for complex

tasks through cross-modal information integration and collaborative learning (Yuan et al. 2025). Its technical principles generally involve three core components: heterogeneous data encoding, cross-modal alignment, and dynamic fusion. First, dedicated networks extract features from each modality's data. Subsequently, contrastive learning or attention mechanisms establish semantic relationships between modalities. Finally, information complementarity is achieved through weighted fusion, gating mechanisms, or other methods tailored to task requirements.

Datasets of source code vulnerability detection

This section addresses RQ1. Code vulnerability detection datasets can be categorized into three primary types according to data source authenticity (Lin et al.

2022): real-world datasets, synthetic datasets, and hybrid datasets. Table 2 summarizes the important information related to the code vulnerability datasets. In addition, there are several code-related datasets (e.g., CodeSearchNet (Husain et al. 2019), CodeNet (Puri et al. 2021), CodeXGLUE (2025)) that can be utilised to assist in enhancing the capabilities of code vulnerability detection tools.

Real-world datasets

Real-world datasets prioritize source authenticity, containing both vulnerability descriptions with corresponding code commit records from the Common Vulnerabilities and Exposures (CVE) and multi-granular code vulnerability annotations. The National Vulnerability Database (NVD) National vulnerability database (2025), maintained by the U.S. National Institute of

Table 2 Overview of the datasets

Real-World			
Dataset	Distribution	Code Language	Granularity
VDSIC (Russell et al. 2018)	1,198,458 : 87,804	C/C++	Function
Devign (Zhou et al. 2019)	14,858 : 12,460	C	Function
Big-Vul (Fan et al. 2020)	253,096 : 11,823	C/C++	Function
CVEfixes (Bhandari et al. 2021)	685 : 22,155	Multi-language(more than 30)	Mixed
D2A (Zheng et al. 2021)	1,276,970 : 18,653	C/C++	Mixed
CrossVul (Nikitopoulos et al. 2021)	13,738 : 13,738	Multi-language(more than 40)	File
ReVeal (Chakraborty et al. 2022)	16,505 : 1,664	C	Function
DiverseVul (Chen et al. 2023)	330,492 : 18,945	C/C++	Function
D-ACT (Pornprasit et al. 2023)	0 : 59,645	Java	File
Real-Vul (Chakraborty et al. 2024)	1,682,713 : 5,528	C/C++	Fine
ReposVul (Wang et al. 2024)	0 : 14,706	Multi-language(4)	Mixed
MegaVul (Ni et al. 2024)	322,168 : 17,380	C/C++	Function
PairVul (Du et al. 2024)	4,314 : 4,314	C/C++	Function
Synthetic			
Dataset	Distribution	Code Language	Granularity
Juliet (Boland And Black 2012)	159,973 : 93,029	C/C++, Java	Function
SARD (Watson et al. 2022)	21,058 : 12,303	C/C++	Function
FormAI (Tihanyi et al. 2023)	54,611 : 57,389	C	File
VGX (Nong et al. 2024)	0 : 150,392	C	Function
Hybrid			
Dataset	Distribution	Code Language	Granularity
NDSS18 (Li et al. 2018)	138,522 : 43,119	C/C++	Code Segment
SARD+NVD (Li et al. 2022)	117,242 : 40,450	C/C++	Mixed
IVSign (Ding et al. 2023)	0 : 8,697	C/C++	Function
W. Qi, et al Qi et al. (2024)	0 : 185,050	C/C++	Function
Promise (Xing et al. 2024)	231 : 69	Java	Mixed

The data is divided into three main categories: real-world datasets, synthetic datasets, and hybrid datasets. "Distribution" indicates the proportion of non-vulnerable data and vulnerability data in the dataset; "Code Language" indicates the type of programming language involved in the dataset; and "Granularity" indicates the annotation granularity of the dataset. Coarse granularity includes: code files, code segments, and code functions. Fine granularity refers to lines of code. Mixed granularity: both coarse and fine granularity

Standards and Technology (NIST), provides standardized storage and analysis of vulnerability data through the Security Content Automation Protocol (SCAP). It enhances CVE vulnerability records with technical details such as severity scores and remediation guidance, forming a comprehensive vulnerability assessment framework.

Early real-world datasets involved relatively few projects. Zhou et al. (2019) extracted commit data from four major C/C++ open-source projects: the Linux Kernel, QEMU, Wireshark, and FFmpeg. They employed the open-source tool Joern to generate ASTs and CFGs, subsequently filtering out functions with evident errors or those that failed to generate valid ASTs or CFGs. The processed functions were then labelled to construct the Devign dataset (FFmpeg+QEMU), comprising 12,460 vulnerable functions and 14,858 non-vulnerable functions, representing a relatively balanced distribution. Saitkat et al. (2022), compiled a real-world dataset, ReVeal, from vulnerabilities reported by developers and users of the Chromium and Debian projects, which contains 1,664 existent and 16,505 nonexistent vulnerability functions, which is closer to the distribution of vulnerability code in the real world.

To address the issues of small sample sizes and limited vulnerability types in early datasets such as Devign and ReVeal, researchers have begun expanding the scale and diversity of C/C++ datasets. Fan et al. (2020) collected a large C/C++ code vulnerability dataset, Big-Vul, from open-source GitHub projects. The dataset primarily captures data from the CVE and CVE-related source code repositories. It includes descriptive vulnerability information from the CVE, such as CVE IDs, severity scores, and summaries. All code repositories were downloaded, and vulnerability-related code changes were extracted using CVE information and their associated GitHub repository links. The Real-Vul dataset (Chakraborty et al. 2024) enhances Big-Vul by selecting the top 10 most vulnerable items from it. Multiple data enhancement techniques were applied to improve dataset diversity, representativeness, and labelling consistency. Similarly, DiverseVul (Chen et al. 2023) is a C/C++ vulnerability source code dataset aggregating data from security issue websites and GitHub. With a volume 60% larger than the previous largest C/C++ dataset, it demonstrates the highest diversity among all comparable datasets. The dataset is constructed by crawling security issue websites and extracting vulnerability-fix commits alongside source code from relevant projects. It comprises 18,945 vulnerable functions (spanning 150 CWEs) and 330,492 non-vulnerable functions, derived from 7,514 commits. Additionally, it encompasses 295 more items than the combined total of all prior datasets.

Naturally, to better serve the task of code vulnerability detection, researchers have not only enhanced the scale of the dataset but also optimised it from a programming language perspective. This approach aims to address the limitation that single-language datasets cannot adequately cover multi-language development scenarios. CVEfixes (Bhandari et al. 2021), developed by researchers at Lund University, are automatically collected and organized from CVE records in the U.S. NVD. This dataset aims to support data-driven security research by providing source code, source code metrics related to CVE fixes in the NVD, repository data, and CVE levels through multi-level abstraction details. It involves over 30 different programming languages. Structured as a relational database, its initial version contains 5,495 vulnerability-fixing commits from 1,754 open-source projects, covering 5,365 CVEs across 180 distinct CWE types. The dataset comprises 18,249 pre- and post-fix source code files, as well as 50,322 functions. In the same year, the CrossVul (Nikitopoulos et al. 2021) dataset primarily sources its data from the NVD, GitHub projects, and Git commits, which include both vulnerable source code files and their patched versions. Distinct from other vulnerability datasets, CrossVul encompasses files written in over 40 programming languages. Each file is linked to a CVE ID and its respective repository. Additionally, the dataset offers a supplementary collection of Git commit messages that serve as security patches.

Recently, researchers have sought to enhance datasets from scalable perspective, addressing issues such as outdated data, lack of cross-process vulnerabilities, and poor scalability inherent in traditional datasets. Wang et al. (2024) proposed an automated data collection framework for code vulnerability detection datasets, addressing challenges such as entangled patches, lack of inter-process vulnerabilities, and outdated patches. They constructed ReposVul, the first repository-level, high-quality vulnerability dataset, by crawling platforms including GitHub, Google Git, and Chromium. The framework comprises three core modules: A vulnerability unravelling module that identifies vulnerability-fixing patches through the combined use of LLMs and static analysis tools; A multi-granularity dependency extraction module capturing inter-procedural call relationships, generating repository-level, file-level, function-level, and line-level metadata for each patch; A trace-based filtering module employing file-path and commit-time trace filters to eliminate obsolete patches and maintain dataset currency. The MegaVul (Ni et al. 2024) dataset, proposed by Ni et al., automatically extracts CVE citation links from the NVD, mines potential vulnerability-fix commits, and downloads

commit metadata and files from five Git-based web services. It employs an enhanced syntax rule-based Tree-sitter for high-quality function extraction, coupled with multiple filters for dataset refinement. Additionally, Ni et al. provide an interface supporting custom filters and Git-based platforms, while MegaVul's implementation can be adapted to other languages (e.g., Java) with minimal code changes.

Takeaway 1

Real-world datasets are primarily extracted from CVE, NVD, and open-source projects. The primary programming language is C/C++, with only a few datasets supporting multiple languages. Sample distribution is generally imbalanced. Annotation granularity predominantly occurs at the function level, with file-level annotation being less common. However, certain limitations exist: datasets reliant on CVE carry the risk of omitting undisclosed vulnerabilities, while static analysis tools are prone to introducing labelling bias. For diverse scenarios, it is recommended to select DiverseVul (covering 150 CWE categories) or CrossVul (supporting 40+ languages) for diversity detection. PairVul/CVEfixes are preferable for remediation research, while MegaVul or ReposVul are recommended for industrial-grade applications.

Synthetic datasets

Synthetic datasets are constructed by artificial construction or computer programs (e.g., rule engines, algorithmic models), which can use deep learning models to analyze real vulnerability patterns and generate synthetic code with similar characteristics.

The current mainstream benchmark datasets for code vulnerability detection each possess distinctive characteristics: The Juliet (Juliet 1.1 for C/C++ and Java) (Boland And Black 2012) is a benchmark collection of C/C++ and Java programs with known defects, containing 57,099 test cases in C/C++ and 23,957 in Java. It covers 181 distinct defect types, including common weaknesses such as authentication and access control, buffer handling, code quality, etc; The Open Web Application Security Project (OWASP) Benchmark (2015) project provides a comprehensive suite of Java test samples for automated evaluation of software vulnerability detection tools. Aimed at addressing the lack of unified evaluation standards for early vulnerability detection tools. The current OWASP Benchmark test suite includes two major versions (V1.1 and V1.2), collectively containing thousands of labelled test cases covering 11 web application vulnerability categories. While V1.1 contained more test cases, they were neither executable nor exploitable, with imbalanced positive and negative samples. V1.2 addresses these limitations by ensuring all test cases are both executable and exploitable while maintaining balanced positive and negative samples; Software Assurance Reference Dataset

(SARD) (Watson et al. 2022), published by the NIST, is designed to support users, researchers, and developers in testing and evaluating security tools and methodologies by providing known security flaws. The dataset aggregates test cases from three sources: industrial production, manual construction, and student-authored tests. Code sample in SARD is classified into three categories: Good (flaw-free), Bad (containing flaws), and Mix (flaws with patches), with flaw locations explicitly labelled. While SARD's test cases originate from diverse sources, they consist exclusively of manually crafted basic samples for testing purposes, exhibiting simpler defect patterns compared to real-world code vulnerabilities.

In recent years, with the advancement of LLMs, researchers have not only begun exploring the security implications of code generated by these models but have also employed LLMs to generate datasets of code vulnerabilities. The FormAI (Tihanyi et al. 2023) dataset consists of compilable C programs generated by LLMs. FormAI-v1 contains 112,000 samples of programs generated by GPT-3.5-turbo. FormAI-v2 extends to 265,000 samples covering multi-model generation results. The dataset uses the form validation tool ESBMC to perform automated vulnerability detection on the generated code, covering common vulnerability types such as memory leaks, buffer overflows, etc. To address the limitations of existing automated sample generation methods, which typically produce noisy samples that fail to meet practical requirements. Nong et al. (2024) proposed VGX, a novel technique for large-scale generation of high-quality vulnerability datasets. Specifically, when presented with a normal programme, VGX first identifies code contexts suitable for vulnerability injection. It then applies editing patterns derived from historical fixes and real-world vulnerabilities to inject the vulnerability into the identified contexts. Deng et al. (2024) developed an approach to address the long-tail distribution challenge in code vulnerability detection datasets. By harnessing the generative and inferential capabilities of LLMs, the method synthesises additional training samples that focus specifically on underrepresented vulnerability types within datasets. The implementation employs GPT-4 as the core generation engine to produce tailored training samples for diverse rare vulnerability categories. Each generated sample undergoes a stringent filtering protocol before being integrated into the detection model's training dataset. Empirical results demonstrate that the LLM-powered data augmentation methodology significantly enhances the model's accuracy in detecting vulnerabilities across multiple vulnerability types.

Takeaway 2

Mainstream synthetic datasets are either manually constructed or programmatically generated, primarily comprising: Juliet covering 181 defect types; SARD providing explicit classification labels but featuring relatively simple sample patterns; FormAI generated and validated via GPT-3.5; OWASP Benchmark focusing on Java Web vulnerabilities (11 categories); and VGX injecting vulnerabilities through historical remediation patterns. Certain limitations exist: SARD samples are manually constructed basic examples with relatively simple defect patterns; OWASP is restricted to web vulnerabilities; VGX relies on predefined contexts; FormAI depends on ESBMC tool validation, potentially overlooking complex vulnerability patterns. For foundational validation, prioritise Juliet or SARD; for generative research, adapt FormAI or VGX. Future efforts should enhance sample complexity.

Hybrid datasets

Hybrid datasets integrate elements from both real-world and synthetic datasets. The NDSS18 (Li et al. 2018) dataset, maintained by NIST, is derived from the NVD and the SARD. Le et al. (2019) compiled the source code into binaries compatible with both Windows and Linux operating systems. The NDSS18 dataset contains a total of 32,281 binary files covering CWE-119 and CWE-322. Similarly, the SARD+NVD (Li et al. 2022) dataset, collected by Li et al. from NVD and SARD, comprises 2,821 real-world programs and 12,329 synthetic programs. NVD-sourced programs contain abundant difference files, with vulnerability line numbers labelled according to code differences

before and after patching. SARD-sourced programs feature extensive labels indicating vulnerability status. The dataset excludes programs shorter than 500 lines as they are overly simplistic for practical vulnerability detection tasks. The IVSign (Ding et al. 2023) dataset, proposed the following year, comprises three principal components. First, the Real-World Vulnerabilities component includes 16 vulnerabilities identified from CVE reports and prior research, covering 20 real-world projects. Second, the Juliet Test Suite component contains numerous small programs, specifically featuring seven CWEs that total 5,383 individual programs. Third, the Online Tutorial component incorporates five practical examples drawn from OWASP's secure programming online tutorial.

To address the sample-related issues in code vulnerability datasets for deep learning detection applications, researchers have proposed targeted data augmentation schemes: Qi et al. (2024) developed a data augmentation technique to tackle the challenges of limited sample size and scarcity of precisely annotated datasets. This approach generates diverse samples at scale by executing transformations that preserve natural semantics; Xing et al. (2024) addressed the dual problems of imbalanced vulnerability category distribution and sparse annotated data in real-world software vulnerability datasets. They designed a phased data augmentation method: first treating vulnerability code as samples and generating additional vulnerability samples through code refactoring to achieve balanced category distribution; then performing hybrid linear interpolation on code embedding vectors, incorporating the resulting interpolated vectors into the training set to expand annotated training data.

Takeaway 3

Hybrid datasets demonstrate unique value in code vulnerability detection by integrating the strengths of real and synthetic data sources. Mainstream hybrid datasets such as NDSS18 and SARD+NVD achieve an organic fusion of real programmes and synthetic samples. SARD+NVD comprises 2,821 real programmes and 12,329 synthetic programmes, primarily covering C/C++ languages, while Promise stands as one of the few hybrid datasets supporting Java. However, these datasets present certain limitations: NDSS18 covers only CWE-119 and CWE-322 vulnerabilities; SARD+NVD's reliance on patch difference annotations may overlook complex contextual factors; and Promise suffers from insufficient data scale. Cross-source validation is recommended using either SARD+NVD or NDSS18. Future work should focus on expanding the proportion of real-world samples and enhancing multi-language support.

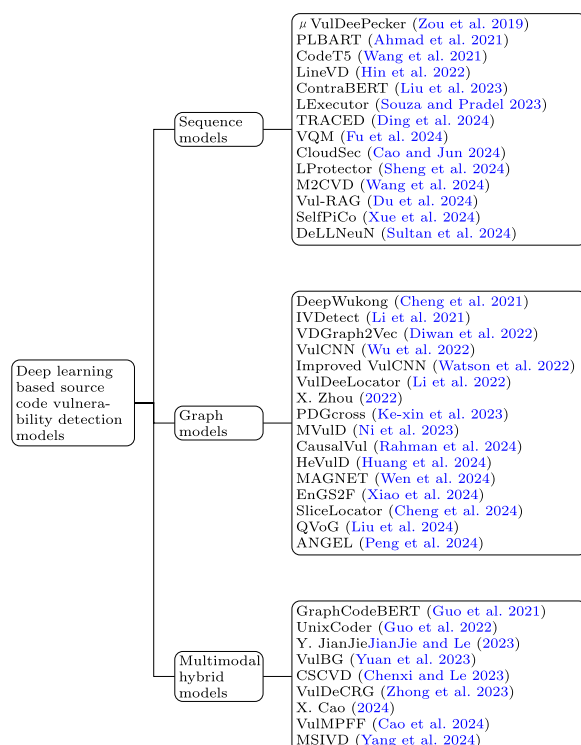


Fig. 4 Deep learning based source code vulnerability detection models

Deep learning-based source code vulnerability detection models

This section focuses on deep learning-based source code vulnerability detection models for RQ2, including model architectures, interpretability analysis, and evaluation metrics. Current technical approaches in this field can be

categorized into three types: sequence-based modelling methods (Chang et al. 2023; Du et al. 2023; Gujar 2024), GNN-based approaches (Sarbakys And Wang 2023; Du et al. 2024), and hybrid models incorporating multimodal features (Shijian 2023; Shouliang 2023). Fig 4 illustrates the architectural framework of mainstream deep learning-based vulnerability detection models.

Sequence models

Sequence code vulnerability detection methods effectively model program structures and logical relationships by transforming source code into high-dimensional semantic vector representations (Kim et al. 2022; Thapa et al. 2022; McCully et al. 2024). Recent advancements in LLMs have endowed these systems with exceptional pattern recognition abilities when processing sequential data (Purba et al. 2023; Mahyari 2024; Mao et al. 2024; Zibaeirad And Vieira 2024), leading researchers to explore their application in code vulnerability detection tasks. Table 3 presents information concerning code vulnerability detection based on sequence models.

Zou et al. (2019) proposed μ VulDeePecker, the first deep learning system for multi-class vulnerability detection. The core architecture of μ VulDeePecker, BLSTM, possesses bidirectional temporal modelling capabilities that capture logical dependencies within code fragments, thereby enabling unified learning of multi-class vulnerability features. With the widespread adoption of Transformer architectures in NLP, sequence models have progressively shifted from BLSTM to Transformer. The core reason lies in the Transformer's self-attention mechanism, which captures global semantic associations

within code. This resolves the long-range dependency forgetting issue inherent in BLSTM when processing lengthy code sequences, such as complex functions or multi-branch logic. PLBART (Ahmad et al. 2021) demonstrates outstanding performance in programme comprehension and generation tasks, achieved through joint denoising pre-training on Java/Python functions and natural language text, thereby establishing a robust foundation for subsequent research in code-related AI applications. Addressing the need for more precise vulnerability localization, Hin et al. (2022) proposed LineVD, a novel deep learning framework that reformulates statement-level vulnerability detection as a node classification problem, enabling finer-grained and more precise identification of vulnerable code segments. Concurrently, efforts to enhance the robustness of pre-trained code models have advanced. Liu et al. (2023) introduced ContraBERT, a method that improves the adversarial robustness of pre-trained models through contrastive learning. Beyond model robustness and vulnerability identification, researchers also focus on bridging the gap between static model representations and dynamic programme behaviour, alongside optimising vulnerability remediation. Ding et al. (2024) propose TRACED to reconcile discrepancies between language models' static representations and programmes' dynamic conduct. Their methodology employs three key elements for model pre-training: source code, executable inputs, and their corresponding execution traces, enabling the model to capture both static code structure and dynamic runtime information. Concurrently, Fu et al. (2024) addressed the challenge of directing model attention towards vulnerable

Table 3 The sequence models

Models	Model infrastructure	Dynamic or static	Dataset	Granularity	Metrics
μ VulDeePecker (Zou et al. 2019)	BLSTM	Static	Code Gadgets	Code Gadget	F1=94.22%
CodeT5 (Wang et al. 2021)	Transformer	Static	Devign	Fine	Acc=64.59%
LExecutor (Souza And Pradel 2023)	Transformer	Dynamic	Real-world Python projects, Stack Overflow code snippets	Function	CC=60.60%
TRACED (Ding et al. 2024)	Transformer	Dynamic	D2A, CodeXGLUE-Devign	Function	Acc=64.00%
CloudSec (Cao And Jun 2024)	Transformer	Static	Juliet	Function	Acc=70.00%
LProtector (Sheng et al. 2024)	Transformer	Static	Big-vul	Codebases	Acc=89.68% F1=33.49%
M2CVD (Wang et al. 2024)	Transformer	Static	Devign, Reveal	Function	Acc=79.94% F1=55.37%
Vul-RAG (Du et al. 2024)	Transformer	Static	A manually curated benchmark suite comprising 60 code functions	Function	F1=79.35%
SelfPiCo (Xue et al. 2024)	Transformer	Dynamic	Real-world Python projects, Stack Overflow code snippets	Function	CC=78.00%
DeLLNeuN (Sultan et al. 2024)	Transformer	Static	DiverseVul	Function	Acc=87.00%

The sequence models are primarily introduced through the following aspects: "Models", "Model Infrastructure", "Dynamic or Static", "Datasets", "Granularity" and "Metrics". Here, "Granularity" denotes the level of detail in code vulnerability detection, while "Metrics" refers to the model assessment techniques employed

code regions during patch generation by proposing the vulnerability patching framework VQM. Drawing inspiration from object detection in computer vision based on Visual Transformers, this framework adapts this vision-centric approach by introducing Vulnerability Queries (VQs).

In recent years, several advances have been made in LLM-based vulnerability detection techniques. Among these contributions, Du et al. (2024) proposed Vul-RAG—an LLM-based vulnerability detection technique implementing a knowledge-level RAG framework. This approach divides vulnerability detection into three sequential stages: knowledge base construction, vulnerability knowledge retrieval, and targeted vulnerability inspection within code snippets, thereby ensuring a systematic and knowledge-driven detection process. However, despite LLMs' exceptional code comprehension capabilities, their practical application in vulnerability detection faces two major challenges: on the one hand, the high computational cost associated with model fine-tuning; on the other, semantic alignment issues directly constrain the accuracy of code vulnerability identification. To address these pressing issues, Wang et al. (2024) proposed the M2CVD framework. This framework combines the powerful semantic analysis capabilities of LLMs with the detection strengths of specialised code models. Through an iterative optimisation mechanism, it further achieves synergistic enhancement between these two model types, effectively mitigating cost burdens and semantic alignment issues. As research in this field progresses, scholars have identified a new limitation in existing LLM-based vulnerability recognition: most studies treat code as pure textual input, failing to capture the contextual information and domain expertise required for accurate detection. To overcome this shortcoming, Sheng et al. (2024) developed LProtector, an automated vulnerability detection system tailored for C/C++ codebases that integrates GPT-4o with RAG. The system employs binary classification for vulnerability identification, enhancing prediction accuracy through advanced retrieval techniques and Chain-of-Thought (CoT) reasoning. Notably, it leverages RAG to integrate domain-specific knowledge, significantly improving its comprehension of complex code structures. Concurrently, Xue et al. (2024) proposed SelfPiCo, a method activating domain knowledge encoded within customised CodeLlama models through minimal lens prompts and CoT reasoning. To further refine code execution accuracy, SelfPiCo incorporates an automated feedback loop: this loop analyses execution trajectories in real-time and dynamically adjusts prediction strategies, enabling the model to adapt to diverse code scenarios.

Recently, researchers have broadened their focus to explore the application of LLMs in cross-file code vulnerability detection (Lin And Mohaisen 2025). Their findings reveal two key insights: on the one hand, LLMs can effectively identify cross-language vulnerability patterns, demonstrating robust generalisation capabilities; on the other hand, their performance is significantly influenced by context window (CW) size—larger CWs directly enhance cross-file vulnerability detection accuracy, whilst quantitative techniques exhibit fluctuating detection results due to variations in model architecture and configuration parameters. These findings not only elucidate critical factors influencing LLM performance in cross-file scenarios but also establish essential technical baselines to guide future research in this direction.

Takeaway 4

From early BLSTM-based model architectures to the current mainstream Transformer architecture, model performance has progressively improved. Transformer-based models can enhance cross-language vulnerability detection capabilities through pre-training, yet differing task requirements have led to architectural diversification. For instance, Vul-RAG integrates domain knowledge via a RAG framework, while M2CVD combines LLM with specialised models to optimise semantic alignment. The dynamic approach TRACED leverages execution traces to improve accuracy, yet faces computational cost challenges; the static method LProtector offers greater scalability but achieves only an F1 score of 33.49%. Overall, the core challenge for sequence models lies in pushing beyond existing capabilities through architectural innovation and retrieval augmentation, while balancing the trade-off between detection granularity (function-level/repository-level) and computational efficiency. With the advancement of LLM technologies, model windows continue to expand, prompting research into larger-scale code vulnerability detection (e.g., file-level, codebase-level).

Graph models

The graph-based code vulnerability detection methodology operates by constructing multidimensional semantic networks that capture program relationships (bin Mohd Illzam et al. 2023; de Kraker et al. 2023; Wen et al. 2023; Yang et al. 2023). This technique comprises two key elements: static program analysis for converting source code into heterogeneous graph representations (encompassing ASTs for structural modelling, CFGs for execution path analysis, and DFGs for variable dependency tracking, etc.), coupled with hierarchical feature learning via GNNs that autonomously identify latent semantic patterns within the encoded graph structures. Table 4 presents information concerning code vulnerability detection based on graph models.

Early research focused on the application of single-graph techniques in code vulnerability detection, proposing a series of solutions addressing various pain points: to enhance the accuracy and efficiency of static

Table 4 The graph models

Models	Model infrastructure	Dynamic or static	Dataset	Granularity	Metrics
DeepWukong (Cheng et al. 2021)	PDG, GNN	Static	SARD, two real-world open-source projects (lua and redis)	Code Fragments	Acc=97.40% F1=95.60%
IVDetect (Li et al. 2021)	PDG, GCN	Static	Devign, Fan et al. (2020), Reveal	Function	F1=48.33%
VulDeeLocator (Li et al. 2022)	AST, BRNN	Static	NVD, SARD	Mixed	Acc=83.90% F1=80.00%
PDGcross (Ke-xin et al. 2023)	PDG, Node2Vec, CDG, DDG	Static	Juliet	File	F1=90.50%
MVulD (Ni et al. 2023)	GNN	Static	Devign, ReVeal, BigVul, CrossVul, CVEFixes	Function	Acc=87.96% F1=35.42%
CausalVul (Rahman et al. 2024)	DAG	Static	Devign, Big-Vul	Function	F1=63.00%
HeVulD (Huang et al. 2024)	AST, CFG, PDG, GNN	Static	SARD	Function	F1=96.40%
MAGNET (Wen et al. 2024)	AST, CFG, DFG, NCS, MHAGNN	Static	Reveal, Devign, Fan et al. (2020)	Function	Acc=82.09% F1=48.46%
EnGS2F (Xiao et al. 2024)	AST, PDG, CRG, GGNN	Static	SARD, NVD, Big-Vul	Function	Acc=97.70% F1=93.20%
SliceLocator (Cheng et al. 2024)	GNN	Dynamic	SARD	Fine	Acc=87%
QVoG (Liu et al. 2024)	CFG, CPG	Static	The GitHub Advisory Database, the QL library of CodeQL	code snippets	Acc=91.70%
ANGEL (Peng et al. 2024)	CPG, GNN, Graph Transformer	Static	Devign, Reveal, Big-Vul	Function	Acc=78.60% F1=43.00%

The graph models are primarily introduced through the following aspects: “Models”, “Model Infrastructure”, “Dynamic or Static”, “Datasets”, “Granularity” and “Metrics”. Here, “Granularity” denotes the level of detail in code vulnerability detection, while “Metrics” refers to the model assessment techniques employed

detection for C/C++ code, DeepWukong (Cheng et al. 2021) integrated deep learning, GNNs, and PDG; IVDetect (Li et al. 2021), building upon similar technologies, further addressed the issue of detection result clarity, optimising fine-grained vulnerability localisation and model interpretability; Cheng et al. (2024) developed SliceLocator to achieve more precise localisation. This approach leverages detectors’ ability to distinguish samples, identifying critical contamination paths by filtering high-weight flow paths. Inspired by deep learning image classification systems’ efficient processing of vast image volumes, Wu et al. (2022) proposed VulCNN to innovate vulnerability detection paradigms.

This approach converts function-level source code into visual representations while preserving programme details, pioneering a vision-driven methodology. Subsequent enhancements to VulCNN (Watson et al. 2022) further resolved localisation inaccuracies, achieving more precise vulnerability pinpointing. Addressing detection challenges posed by cross-file function calls, Xiong et al. (2023) introduced the novel PDGcross graph representation. This aggregates multi-file call relationships from single entry points to construct dependency graphs. After conversion into feature matrices via the Node2Vec algorithm, an LSTM network trains the model, forming an integrated solution

Table 5 The multimodal hybrid models

Models	Model infrastructure	Dynamic or static	Dataset	Granularity	Metrics
Jian Jie and Le (2023)	TextCNN, BLSTM-CNN, GCNN, BLSTM	Static	Juliet, CodeXGLUE	Function	Acc=79.43% F1=74.16%
VulBG (Yuan et al. 2023)	TextCNN, ASTGRU, VulCNN, CodeBERT	Static	Devign, Reveal	Function	F1=46.30%
CSCVD (Chenxi And Le 2023)	CFG, CAT, BLSTM	Static	CodeXGLUE	Function	Acc=63.00%
VulDeCRG (Zhong et al. 2023)	CPG, GGNN, Word2vec, BLSTM	Static	Big-Vul, Devign	Function	F1=65.60%
VulMPFF (Cao et al. 2024)	AST, CPG, GNN, Word2Vec, BLSTM	Static	Big-Vul, Devign, Reveal	Function	Acc=83.90% F1=51.30%
MSIVD (Yang et al. 2024)	AST, GNN, LLM	Static	BigVul, PreciseBugs	Function	F1=70.00%

The multimodal hybrid models are primarily introduced through the following aspects: "Models", "Model Infrastructure", "Dynamic or Static", "Datasets", "Granularity" and "Metrics". Here, "Granularity" denotes the level of detail in code vulnerability detection, while "Metrics" refers to the model assessment techniques employed

combining cross-file PDG with deep learning to resolve inter-file vulnerability detection. To overcome limitations in existing research concerning insufficient utilisation of images and underlying graph structures, Ni et al. (2023) developed MVulD. This integrates the pre-trained UniXcoder model, GNNs, and computer vision techniques to extract function features across multiple dimensions. The following year, Rahman et al. 2024 proposed CausalVul to enhance model robustness and reliability by incorporating causal reasoning into deep neural networks.

Current graph-based code vulnerability detection is evolving from single-graph structures towards multi-graph heterogeneous fusion. This represents a core direction for overcoming the limitations of traditional graph models' structural information, achieving dual improvements in detection accuracy and generalisation capability. Early graph models predominantly relied on single-type graph structures. While these monolithic graphs could focus on specific dimensions of code relationships, they struggled to encompass the multidimensional information required for vulnerability detection. To address this limitation, researchers have begun leveraging multi-graph heterogeneous fusion to harness the core strengths of diverse code graph structures. Huang et al. (2024) introduced HeVulD, a novel vulnerability detection framework for C/C++ programs. The

proposed solution incorporates two node definition techniques along with a key node-centric program slicing approach, specifically designed to construct heterogeneous graphs from source code. To address the same problem, Wen et al. (2024) proposed MAGNET, an attention graph learning model based on heterogeneous relational metapaths. On the other hand, Xiao et al. (2024) proposed EnGS2F, which employs an augmented graph structure to represent learning and improves the efficiency of capturing vulnerability information.

As research progresses, Liu et al. (2024) investigated the performance and scalability issues faced by graph-based code vulnerability analysis tools. Addressing these challenges, they introduce QVoG, a static analysis platform incorporating three crucial improvements: a compressed CPG representation that reduces graph size while retaining critical information, a specialized declarative query language based on CPG for efficient vulnerability queries, and embedded machine learning components that improve vulnerability detection's generalization capacity. Peng et al. (2024) proposed ANGEL to mitigate the interference of noise information and resolve the challenge of capturing long-range dependencies in graph-based code vulnerability detection methods. The proposed method integrates hierarchical graph refinement with context-aware representation learning. It systematically eliminates

redundant information while preserving structural integrity and effectively addressing the long-range dependency problem.

Takeaway 5

Research on code vulnerability detection based on graph models exhibits three distinct characteristics: Firstly, mainstream approaches employ joint modelling of CPGs and GNNs. Secondly, heterogeneous graph modelling has emerged as a breakthrough direction, with HeVulD and MAGNET enhancing detection capabilities through differentiated nodes and multi-level attention mechanisms respectively. Finally, existing research faces a core trade-off between accuracy and scalability, with lightweight solutions (e.g., CausalVul) exhibiting limited generalisation capabilities.

Multimodal hybrid models

Existing technologies have their advantages and disadvantages. Text-based techniques demonstrate scalability but suffer from inaccuracy due to their lack of procedural semantics. Conversely, graph-based methods provide accuracy at the cost of scalability. The multimodal hybrid approach for code vulnerability detection establishes an integrated representation system that captures both surface-level syntactic features and deep program logic mainly by leveraging the complementary strengths of sequence modelling and graph-based structural analysis (Sun et al. 2023; Tang et al. 2023; Yuan et al. 2024). Table 5 presents information concerning code vulnerability detection based on multimodal hybrid models.

Guo et al. (2021) proposed GraphCodeBERT to overcome the limitation of pre-trained models treating code fragments solely as token sequences while neglecting their intrinsic structural properties. Unlike approaches relying on ASTs, this model captures variable relationships during pre-training by leveraging semantic-level structures such as data flow. These semantic structures maintain conciseness by avoiding excessive depth, thereby effectively enhancing model efficiency. Similarly, Yang et al. (2023) addressed the critical limitation of existing source code vulnerability detection methods—where unimodal approaches (either pure sequence or graph-based) fail to effectively capture code’s multidimensional features—by proposing the CSCVD method. This approach employs a three-stage architecture to fuse semantic features with contextual dependencies, thereby overcoming the detection bottleneck inherent in unimodal methods.

Furthermore, Liang et al. (2024) developed a detection framework for mobile application vulnerabilities by integrating CPG representations with adaptive Graph Neural Networks (GNNs). This framework follows a three-stage process: first converting source code into CPG format, then vectorising nodes via CodeBERT embeddings, and

finally extracting features using a GNN architecture combining Transformer-based adaptive attention with GCNs to enhance vulnerability detection efficacy.

Cao et al. (2024) identified that existing deep learning-based vulnerability detection methods rely on a single intermediate code representation (IRC), severely limiting detection capabilities. Consequently, they developed the multi-perspective code feature fusion framework VulMPFF. This approach systematically extracts IRC representations from three complementary dimensions: sequential code patterns, lexical-syntactic relationships, and graph-based structural features. By employing a tri-modal representation strategy, it comprehensively captures vulnerability information, with the synergistic effect of multiple IRCs significantly enhancing detection accuracy.

Although Transformer-based language models have demonstrated improved performance in code vulnerability detection, token-only training models exhibit two critical limitations: their inability to capture vulnerability semantic features and code data flow structures. To address these issues, Yang et al. (2024) developed the MSIVD framework, which integrates multitask LLM with GNNs to CFG encoding, enabling end-to-end vulnerability detection through CoT prompting and self-supervised fine-tuning.

Takeaway 6

Current research on code vulnerability detection based on multimodal hybrid models primarily integrates the strengths of sequence models (e.g., BLSTM, Transformer) and graph models (e.g., GNN, CPG). Common model architecture combinations include: “CPG + GNN + BLSTM” and “CPG + GNN + Transformer”. These models confront a fundamental trade-off between multimodal information gain and computational efficiency. High-performance solutions (e.g., VulMPFF) sacrifice real-time capability for accuracy, whilst lightweight designs (e.g., GraphCodeBERT) balance efficiency and structural complexity through data-flow pre-training.

Interpretability

The intrinsic black-box nature of deep learning models constitutes a fundamental challenge to interpretability. This interpretability limitation primarily stems from the complex hierarchical architecture and non-linear connectivity mechanisms of neural networks, which obscure the internal decision-making processes. Although significant progress has been made in deep learning interpretability research (Chen et al. 2023; Gao And Guan 2023; Yu And Ananiadou 2024; Hassija et al. 2024; Hosain et al. 2024; Ganz 2024), investigations specifically targeting code vulnerability detection remain at a nascent stage. Current studies typically categorize the interpretability of code vulnerability detection models into two distinct

dimensions based on granularity (Sotgiu et al. 2022); macro-level global interpretability (Li et al. 2020) and micro-level local interpretability (Marchetto 2024).

Global interpretability

Global interpretability seeks to demystify the internal operations of deep learning models by systematically analysing their decision-making patterns across entire datasets, thereby illuminating fundamental inference mechanisms (Che et al. 2015; Xu et al. 2018; Tan et al. 2018). This approach adopts a macro-analytical perspective to investigate systematic relationships between input features and model outputs, such as characteristic importance (Walia et al. 2022; Nguyen et al. 2023) and causal structure modelling.

Characteristic importance

Recent advancements in interpretability for deep learning-based code vulnerability detection have made some progress. The SHAP method (Lundberg And Lee 2017) quantifies feature importance scores, providing an intuitive representation of how different input features contribute to the model's final decisions. This methodology ultimately reveals how models intrinsically integrate multidimensional information to achieve generalized predictive capabilities. Graph-based interpretability techniques primarily improve model transparency through three key mechanisms: assigning numerical weights to graph edges (Ying et al. 2019), computing node importance scores (Pope et al. 2019), and evaluating the contribution of GNN traversal paths (Schnake et al. 2022). Nevertheless, these GNN interpretation approaches continue to encounter fundamental limitations in extracting fine-grained vulnerability information. Ding and colleagues (Ding et al. 2023) devised the IVSign scheme that synergistically integrates pattern-driven and data-driven methodologies: the pattern-driven component utilizes static analysis to extract vulnerability tracking information from data dependencies, whereas the data-driven component adopts a Transformer-RNN architecture to encode tracking data and produce interpretable vulnerability signatures. Separately, Marchetto et al. (2024) incorporated the SHAP interpretation framework into both VulDeePecker and JavaBert models, enabling accurate localization of vulnerable code segments and their related statements. Furthermore, Cheng et al. (2024) introduced SliceLocator, a tool that examines the detector's discriminative characteristics between vulnerable and non-vulnerable samples. It methodically identifies

potential vulnerability-triggering statements and subsequently pinpoints critical data flow paths through integrated taint analysis techniques.

Causal structural modelling

Cao et al. (2024) introduced Coca, a generic framework designed with two primary objectives: first, improving the robustness of GNN-based vulnerability detection models to prevent spurious explanations; and second, generating concise yet effective explanations for identified vulnerabilities. The Coca framework comprises two core components: the coach and the interpreter. The coach component employs combinatorial contrast learning to develop a detection model resilient to random perturbations, whereas the interpreter component utilizes two-view causal reasoning to identify and extract the most determinative code statements underlying the detected vulnerabilities, thereby providing actionable explanations.

Takeaway 7

Global interpretability approaches for code vulnerability detection models analyse decision patterns across entire datasets from a macro perspective, primarily encompassing feature importance and causal structure modelling: Feature importance methods prioritise intuitive visualisation of how different input features influence model decisions, and can be integrated with existing detection models (e.g., VulDeePecker, JavaBert); Causal structure modelling, conversely, prioritises eliminating confounding factors to generate precise, actionable vulnerability explanations, offering superior robustness against perturbations. However, global interpretability approaches exhibit notable limitations. GNN-related techniques within feature importance struggle to extract fine-grained vulnerability information, failing to precisely pinpoint critical code segments and data flow paths triggering vulnerabilities. Furthermore, there is an overall lack of integration with local interpretability, making it difficult to simultaneously satisfy developers' dual demands for explanations of both the model's overall logic and specific vulnerability details.

Local interpretability

Local interpretability focuses on deconstructing the "black box" logic of a model's decision-making for individual samples by analyzing causal relationships between specific inputs and outputs (Bhattacharjee et al. 2023; Kroeger et al. 2023; Zhou et al. 2024; Luo et al. 2024). Common approaches include: attentional mechanisms, natural language interpretation (Turpin et al. 2023), and counterfactual interpretation. It aims to reveal why the model generated a particular prediction for a given instance, essentially answering the question "What features does the model focus on here" This is achieved through techniques like constructing local surrogate models using LIME (Pedersen et al. 2017) or visualizing

critical regions in images via gradient-based heatmaps. As regulatory requirements for algorithmic transparency increase, local interpretability is becoming a crucial technical foundation for maintaining both model performance and ethical accountability.

Attention mechanisms

Li et al. (2024b) employ attention mechanisms for model interpretation, leveraging their inherent ability to capture token correlations, a key interpretability feature of Transformer architectures. Recent research has demonstrated significant advancements in utilizing attention mechanisms to enhance language model interpretability. Notably, studies confirm that self-attention mechanisms in large language models effectively capture both syntactic and semantic information in code through attention distribution analysis. Ding et al. (2023) developed VulANalyzeR, a new model for automated binary vulnerability detection that performs: classification of common weakness types, and root-cause analysis. The model combines sequential processing via cyclic cells with topological analysis through graph convolution learning to simulate program execution. Crucially, attention mechanisms permeate the entire architecture, visually demonstrating how different instructions and program states contribute to classification decisions. Additionally, VulANalyzeR employs multi-task learning to classify specific vulnerability types, achieving the dual benefits of enhanced explainability and accelerated zero-day vulnerability patching.

Natural language interpretation

Li et al. (2021) proposed IVDetect, which focuses on improving the accuracy and interpretability of vulnerability localization. The system detects vulnerabilities using AI techniques and generates detailed descriptions of vulnerable code with the aid of intelligent assistants. During the vulnerability detection phase, the method analyses both data and control dependencies, encompassing vulnerable statements and their associated contextual information. It employs interpretable AI techniques to produce fine-grained analysis results that include PDG substructures and relevant vulnerability statements. The LLMVulExp framework developed by Mao et al. (2024) utilizes automated design to enable LLMs to perform dual functions of vulnerability detection and explanation. The framework employs prompt engineering to annotate explanation content and incorporates LoRA instruction tuning to equip the model with vulnerability type identification and causal analysis capabilities. By integrating CoT technology to focus on key code segments, the

framework significantly improves both vulnerability detection accuracy and explanation completeness.

Counterfactual explanation

Building upon recent advancements in AI counterfactual reasoning, Chu et al. (2024) proposed CFExplainer, a counterfactual-based interpreter specifically designed for GNN vulnerability detection. Distinct from conventional fact-based reasoning interpreters, CFExplainer's core functionality involves identifying the minimal required perturbations in input code graphs that would alter the model's predictions. This approach enables precise localization of vulnerability root causes, thereby empowering developers to implement targeted remediation measures.

Takeaway 8

Local interpretability methods for code vulnerability detection models deconstruct decision logic from a microscopic perspective. Their core feature lies in focusing on the causal relationship between input and output for individual samples, aiming to clarify the model's focus on specific vulnerabilities. These methods encompass three categories: attention mechanisms, natural language explanations, and counterfactual explanations. Attention mechanisms (e.g., VulANalyzeR) capture correlations between code tokens; LLMVulExp employs prompt engineering and LoRA tuning to enhance vulnerability classification and causal analysis through natural language explanations; counterfactual explanations (e.g., CFExplainer) precisely pinpoint root causes by identifying minimal perturbations in input code graphs. Each approach possesses distinct characteristics: attention mechanisms excel at intuitively presenting the relational weights of code elements; natural language explanations prioritise outputting analysis results in human-readable formats; counterfactual explanations demonstrate superior precision in pinpointing vulnerability root causes. However, local explainability methods exhibit notable limitations, such as attention mechanisms being susceptible to bias from redundant syntactic information within code.

Evaluation metrics

Current research on deep learning-based code vulnerability detection primarily employs standard evaluation metrics commonly used in classification tasks. Fundamental classification metrics including Accuracy (Acc), Precision (P), Recall (R), and F_1 score, along with comprehensive performance measures like the Area Under the Receiver Operating Characteristic Curve (AUC-ROC), constitute the core evaluation framework. These metrics effectively evaluate model performance in both binary classification (vulnerability presence/absence) and multiclass classification (various vulnerability types) scenarios. Furthermore, considering the unique requirements of code vulnerability detection, researchers often incorporate specialized metrics such as FPR

and False Negative Rate (FNR) to provide a more comprehensive assessment of model reliability in practical applications. Of course, there are also auxiliary evaluation metrics for code vulnerability detection, such as CC. Specific evaluation metrics are as follows:

- True Positive(TP): The number of samples where the model detects as a vulnerability and actually is also a vulnerability.
- False Positive(FP): The number of samples that the model detects as vulnerabilities and are actually non-vulnerable.
- True Negative(TN): The number of samples that the model detects as non-vulnerable and actually is also non-vulnerable.
- False Negative(FN): Number of samples where the model detects a non-vulnerability that is actually a vulnerability.
- Acc: The ratio of the number of samples which the model detects correctly to the total number of samples in the test data. Formula: 1

$$Accuracy = \frac{TP + TN}{TP + FP + TN + FN} \quad (1)$$

- Precision: The percentage of real vulnerability samples among those detected as vulnerabilities by the model. Formula 2

$$Precision = \frac{TP}{TP + FP} \quad (2)$$

- Recall: The percentage of all vulnerability samples of the test data that are predicted as vulnerabilities by the model. Formula 3

$$Recall = \frac{TP}{TP + FN} \quad (3)$$

- F_1 score: Metrics that enable a comprehensive assessment of the precision and recall of model detection. Formula 4

$$F_1 = \frac{2 \times Precision \times Recall}{Precision + Recall} \quad (4)$$

- FNR: The ratio of samples detected by the model as non-vulnerable that are actually vulnerable to all vulnerable samples in the test set. Formula 5

$$FNR = \frac{FN}{TP + FN} \quad (5)$$

- FPR: The ratio of samples that the model detects as vulnerabilities and are actually non-vulnerable to all non-vulnerable samples in the test set. Formula 6

$$FPR = \frac{FP}{TN + FP} \quad (6)$$

- ROC: A curve drawn in the plane with FPR as the horizontal coordinate and TPR as the vertical coordinate.
- AUC: The area under the ROC curve enclosed with the axes.
- CC: This is a core metric for measuring the adequacy of software testing, quantifying the proportion of code executed during the testing process.

Takeaway 9

Current deep learning-based source code vulnerability detection primarily focuses on classification tasks. It is worth noting that since code vulnerability detection is essentially an unbalanced classification problem, researchers often need to pay special attention to the model's ability to detect a small number of classes, and thus the Recall rate and F1 value are usually regarded as more important evaluation criteria.

The study of application scenarios

This section investigates Research Question 3 through three distinct application scenarios in code vulnerability detection: mobile applications (Lakshmanarao And Shashi 2022; Thangavel et al. 2022; Wang et al. 2024), smart contracts (Tang et al. 2023; Dong and Li 2023; Liu et al. 2024), and code generation (Basic And Giarretta 2024; Tihanyi et al. 2025). In the mobile application domain, 4.3 billion global smartphone users face security vulnerability risks stemming from rapid iteration cycles. Deep learning proves effective in identifying native defects such as hard-coded credentials within mobile applications (Doll et al. 2025). Smart contracts have seen widespread adoption across numerous sectors including finance, supply chains, and the Internet of Things. However, the surge in smart contract security incidents in recent years has resulted in significant economic losses (CISA 2025); meanwhile, AI-generated code in code generation scenarios has exhibited security vulnerabilities (Jesse et al. 2023).

Mobile application

Mobile applications are software programmes designed for mobile terminals, delivering interactive services via wireless networks. Their code security is pivotal to safeguarding user privacy, corporate assets, and ecosystem stability (Sarkar et al. 2019; Garg And Baliyan 2021; Gao et al. 2021). Common programming languages used in development include Java (Android development), JavaScript (front-end interaction), Kotlin (Sharma 2023) (emerging Android standard), Swift (dominant

for iOS/macOS), and Dart (cross-platform development), each exhibiting significant differences in language characteristics (Kenesova et al. 2022). The 2024 OWASP Top Ten Mobile Application Risks (Owasp mobile security 2025b) encompassed: inadequate credential management, supply chain vulnerabilities, authentication flaws, and insecure data storage. Furthermore, the introduction of AI agents presents new challenges including permission abuse, data privacy breaches, and systemic security threats (Ding 2024; Hu et al. 2025; Wu et al. 2025).

Mobile application code security detection faces multiple challenges across different dimensions. First, regarding the operating environment, the coexistence of heterogeneous hardware architectures (e.g., ARM, x86) and diverse operating systems (e.g., Android, iOS) creates difficulties in establishing unified detection standards. Second, variations in development approaches (e.g., HTML5-based web applications, native applications utilizing system APIs, and hybrid architectures) significantly increase detection complexity (Zhen et al. 2023). Third, at the technical level: static analysis methods are vulnerable to code obfuscation techniques; dynamic testing suffers from incomplete execution path coverage; hybrid detection approaches encounter exponentially growing algorithmic complexity; and AI-driven behavioural analysis faces constraints due to high data annotation costs and limited model generalization. Furthermore, supply chain security presents dual threats: potential hardware backdoors and risks of compromised development toolchains.

Smart contract

Smart contracts are automated program codes stored on a blockchain that execute transactions or operations automatically when predetermined conditions are met. At their core, they represent intelligent execution protocols that transcend traditional electronic contracts (Zhang et al. 2022; Zou et al. 2025; Bresil et al. 2025). Mainstream development languages include (Chu et al. 2023) : Solidity (Parizi et al. 2018; Wu et al. 2022; Zhou et al. 2023) (Ethereum's standard language, supporting complex logic), Vyper (emphasising security and readability), Rust (Mishra et al. 2024) (Solana's preferred choice, ensuring memory safety), and Move (designed specifically for digital assets, guaranteeing asset uniqueness). According to the OWASP 2025 report, smart contracts face ten major risks (Owasp smart contract security 2025c), such as access control vulnerabilities, re-entrancy attacks, and integer overflows. These vulnerabilities may lead to asset manipulation, fund loss, or system paralysis, posing a severe threat to blockchain security.

The detection of smart contract security vulnerabilities faces multiple challenges (Wu et al. 2024; Yu And Lee 2025). Business logic-based vulnerabilities present

particular risks due to their concealed nature, economic model design flaws (including miscalculations of clearing thresholds and pledge rates) demand specialized financial engineering expertise that exceeds traditional code auditing capabilities. Concurrently, cross-protocol attacks are growing increasingly sophisticated, as attackers exploit DeFi's composability to construct complex call chains (e.g., nested reentrancy attacks) (Zhang 2024); Existing detection methods exhibit fundamental limitations. Static analysis cannot adequately address runtime state dependencies and cross-contract call paths. Dynamic testing suffers from path explosion issues and environment discrepancies. While formal verification encounters difficulties in accurately modelling intricate financial contracts; The toolchain presents additional obstacles. Multi-chain ecosystem fragmentation necessitates customized detection solutions, cross-chain bridge vulnerability validation lacks standardization, and on-chain data analysis grapples with processing massive datasets at prohibitive costs; Most alarmingly, vulnerabilities continue to evolve rapidly. Reentrancy attacks have developed new variants including callback injection, while security rule updates struggle to keep pace with attack innovation. Furthermore, AI-generated code risks introducing novel logic flaws stemming from training data biases (Yong et al. 2025).

Code generation

LLM code generation technology triggers automated programming through natural language descriptions or code snippets, significantly enhancing development efficiency (45% of developers utilise AI tools, with 34% relying on ChatGPT (CSDN 2025)). However, research indicates significant security risks in AI-generated code (Jesse et al. 2023), such as 35.8% of GitHub Copilot-generated code snippets containing vulnerabilities (Fu et al. 2023). Model output quality strongly correlates with training data (Jiang et al. 2024; Espejel et al. 2023). Training in a single language (e.g., Python) improves code accuracy, but multilingual mixing may cause feature confusion. The quality of code generated by different large language models across various programming languages varies considerably. Whilst some models demonstrate reasonable security, vulnerabilities persist and require enhanced protective measures (Kharma et al. 2025). In 2025, OWASP identified ten major risks associated with LLM applications (Owasp llm applications security 2025a; Yang et al. 2024; Liu et al. 2023), including prompt injection, sensitive information disclosure, supply chain vulnerabilities (Le et al. 2024a), and model hallucinations. These challenges threaten the security and reliability of AI-generated code.

Detecting vulnerabilities in code generated by LLMs presents multiple challenges. First, the high degree of covertness (Negri-Ribalta et al. 2024; Mohsin et al. 2024), malicious instructions are nested so that the attack code becomes deeply integrated with business logic. Second, there are limitations in contextual understanding, the system struggles to properly interpret business rules, environmental constraints, and version compatibility requirements (Zhang et al. 2023, 2024). Third, the user base is mixed (including both professionals and non-professionals), and the software development cycle is compressed, resulting in the rapid accumulation of vulnerable code. Fourth, weak defence mechanisms present issues including training data contamination (Vadisetty et al. 2023), difficulty in responsibility tracing (across training, generation and deployment phases), and repair risks.

Takeaway 10

Through systematic analysis of three key scenarios-mobile applications, smart contracts, and code generation-this study reveals distinct programming language ecosystems, vulnerability characteristics, and detection challenges across different domains. Programming language distribution reveals a multi-language ecosystem centred on Java/Kotlin/Swift for mobile applications, specialised languages like Solidity/Vyper/Rust for smart contracts, and code generation quality strongly correlated with training data language preferences. High-frequency vulnerability patterns exhibit scenario specificity: mobile apps face supply chain risks and privacy leaks, smart contracts are vulnerable to economic model attacks, while code generation encounters novel threats such as prompt injection and data pollution. Regarding code vulnerability detection challenges, mobile applications must address compatibility issues across heterogeneous environments, smart contracts require countermeasures against cross-protocol composite attacks, and code generation faces difficulties in accountability tracing due to the stealthiness of malicious instructions. These differences underscore the necessity for detection technologies tailored to specific development scenarios.

Challenges and future work

Research has demonstrated that deep learning techniques are transforming the technical landscape of source code vulnerability detection, with their applications already yielding some progress. Despite this, the field continues to confront several critical challenges. These challenges primarily manifest in the disparity between technological advancements and their practical application. At the data level, obtaining high-quality datasets remains problematic. Regarding model capabilities, a gap persists between deep learning-based multimodal code vulnerability detection and actual vulnerability identification requirements, coupled with the need to enhance model interpretability. Furthermore, different application scenarios present distinct challenges in code vulnerability detection due to varying business requirements. In this section, we will conduct a systematic analysis of these core challenges across all dimensions and investigate potential

solutions to facilitate more effective implementation of source code vulnerability detection technologies.

Challenge 1: How to build a large number of high-quality real datasets?

Current code vulnerability detection datasets face challenges on multiple fronts: Firstly, the quality and applicability of datasets are constrained. While real-world datasets (such as Devign and MegaVul) closely resemble practical scenarios, they predominantly rely on CVE/NVD and open-source projects. This approach risks overlooking undisclosed vulnerabilities and exhibits a bias towards C/C++, with insufficient multi-language support (only a few datasets like CrossVul cover multiple programming languages). Annotation granularity is concentrated at the function level, with limited file-level coverage. Synthetic datasets (e.g., SARD, OWASP Benchmark) offer controllable samples but suffer from simplistic patterns and narrow scenarios (e.g., OWASP focuses solely on Java web vulnerabilities). Hybrid datasets (e.g., NDSS18, SARD+NVD) combine strengths yet exhibit limitations: narrow vulnerability type coverage (e.g., NDSS18 includes only CWE-119/322) and low proportions of real-world samples. Secondly, data annotation and distribution present shortcomings (Croft et al. 2023; Shihab And Alanezi 2024). Annotation relies on costly expert verification, while static analysis tools readily introduce bias. Concurrently, vulnerabilities exhibit a “long-tail distribution” (abundant high-severity samples, scarce marginal samples).

Future research may be enhanced in the following areas: Firstly, constructing high-quality, multi-dimensional datasets by supplementing real-world data through multiple channels, while combining static analysis tools with LLMs to improve annotation accuracy and efficiency. Simultaneously, expand multilingual coverage (e.g., transferring C/C++ vulnerability characteristics to Java and Python) and enrich annotation granularity. Secondly, optimise data balance and scenario adaptability by employing variant testing (variable substitution, control flow perturbation) and LLM-generated synthetic samples for long-tail vulnerabilities. This approach enhances the proportion of real samples and vulnerability type coverage within hybrid datasets.

Challenge 2: How to improve deep learning-based multimodal code vulnerability detection?

Current multimodal code vulnerability detection models, whilst effectively addressing the shortcomings of single-text or single-graph modalities in capturing syntactic features and program logic by integrating the strengths of sequence models (e.g., BLSTM, Transformer) and graph models (e.g., CPG, GNN), still face several core challenges: Firstly, insufficient depth in modality fusion. Existing models predominantly remain

at the feature-level, employing simple concatenation or parallel input approaches. This fails to fully exploit the intrinsic connections between sequential semantics (e.g., code token associations) and graph structures (e.g., CFG, DFG), often resulting in redundant modal information or the loss of critical interaction features. Secondly, inadequate modelling capabilities for lengthy code and cross-module dependencies. Text models struggle with excessively long code snippets due to contextual window constraints, while graph models fail to effectively capture cross-module dependencies, resulting in vulnerabilities being overlooked in complex projects; Thirdly, efficiency and performance prove difficult to reconcile. High-performance models (e.g., VulMPFF), while integrating multi-dimensional IRC to enhance detection accuracy, incur significantly increased computational overhead at the expense of real-time capability. Conversely, lightweight models (e.g., GraphCodeBERT), though optimised for efficiency, suffer accuracy degradation in complex nested vulnerability scenarios due to insufficient capture of semantic features.

Future research will focus on the following aspects: Firstly, exploring deep modality fusion mechanisms based on cross-modal attention or causal relationship modelling to establish dynamic mapping relationships between sequence semantics and graph structures. For instance, utilising token dependencies captured by Transformers as weight adjustment factors for GNN node features to achieve collaborative optimisation of modal information, thereby reducing redundancy and enhancing key features. Secondly, optimising long-code and cross-module modelling techniques. This involves reducing input dimensionality through intelligent code segmentation and key feature summarisation, while designing cross-module dependency graph construction methods. Attention mechanisms can be employed to focus on core associative pathways, thereby enhancing adaptability in complex scenarios; Thirdly, constructing architectures balancing efficiency and performance. This involves simplifying graph computations through lightweight GNNs, introducing modal feature selection mechanisms (such as attention-weighted selection of high-contribution features), or employing knowledge distillation techniques to reduce multimodal model size. These approaches enhance real-time capabilities while maintaining detection accuracy.

Challenge 3: Practical deployment of deep learning-based code vulnerability detection models.

Current mainstream deep learning source code vulnerability detection models still face multiple challenges when deployed in practical industrial scenarios. Firstly, in terms of computational cost, resource consumption varies across different model architectures: sequence models

such as TRACED rely on joint pre-training of code execution traces and input data, which enhances detection accuracy but requires substantial computational resources during real-time operation to process dynamic execution data, making them ill-suited for resource-constrained edge computing scenarios; Graph-based models like HeVulD and EnGS2F, while enhancing structural semantic capture through heterogeneous graph reinforcement, suffer from slow inference speeds due to complex node computations involved in graph construction and GNN hierarchical feature learning. This renders them unsuitable for real-time vulnerability scanning of large-scale codebases. Furthermore, data transformation and cross-modal alignment operations during feature fusion in multimodal hybrid models further increase computational overhead, diminishing their practicality in low-cost deployment scenarios. Secondly, model maintenance proves challenging. Vulnerability types evolve with programming language iterations and attack technique advancements, necessitating frequent updates to training data and parameters for existing models. Thirdly, current models remain vulnerable to malicious code obfuscation attacks. While sequence models like ContraBERT enhance resilience through contrastive learning and data augmentation, existing defences exhibit significant shortcomings against techniques such as code obfuscation or stealthy vulnerability injection. Furthermore, while multimodal models integrate multi-source features, vulnerabilities in cross-modal feature fusion may be exploited. Attackers can degrade overall detection accuracy by deliberately corrupting data from a single modality. Defensive strategies against multimodal adversarial attacks remain exploratory in current research, with no mature solutions yet established.

Future research must overcome these obstacles through three avenues: computational cost optimisation may explore lightweight model designs, reducing computational overhead while maintaining accuracy by pruning redundant graph nodes or compressing multimodal feature dimensions; model maintenance efficiency could be enhanced by establishing dynamic knowledge update mechanisms, leveraging RAG technologies (e.g., Vul-RAG) to integrate real-time vulnerability databases (e.g., CVE), enabling incremental model knowledge updates and reducing full-scale fine-tuning costs; Enhancing adversarial robustness requires strengthening multi-dimensional defence strategies. This entails expanding data augmentation techniques to cover more adversarial scenarios (e.g., generating samples after code obfuscation), while also incorporating causal reasoning (e.g., the DAG structure in CausalVul) to improve the model's fault tolerance against attack perturbations.

Challenge 4: How can we continue to improve the interpretability of deep learning-based code vulnerability detection?

Research into the interpretability of current code vulnerability detection models continues to face multiple core challenges. Firstly, there is a lack of fine-grained explanatory capability at the global level, making it difficult to precisely pinpoint the critical code segments and data flow paths that trigger vulnerabilities. While causal structure modelling (e.g., Coca) can enhance model robustness, it remains insufficient for decomposing causal chains in complex nested vulnerability scenarios. Secondly, local explanations suffer from technical limitations. Attention mechanisms, while capable of capturing relationships between code tokens, are susceptible to interference from redundant syntactic information, leading to interpretative bias. Natural language explanations (e.g., IVDetect, LLMVulExp) rely on manually designed prompt engineering or instruction tuning, resulting in insufficient descriptive completeness for niche vulnerability types. Counterfactual explanations (e.g., CFExplainer), though capable of pinpointing root causes, exhibit poor adaptability to textual code or binary code. Thirdly, there exists a disconnect between global and local explanations. Existing research predominantly focuses on either dimension in isolation, lacking a unified explanatory framework capable of integrating macro-level decision patterns with micro-level sample logic.

Future research may be enhanced in three areas: Firstly, advancing fine-grained and multimodal explanation techniques. For global explanations, integrating static programme analysis may optimise feature importance calculations, enabling hierarchical explanations across features, code statements, and vulnerability types. For local explanations, counterfactual generation algorithms spanning input modalities should be developed, incorporating the contextual understanding capabilities of large language models to enhance the accuracy and generality of natural language explanations. Secondly, construct a collaborative explanation framework integrating global and local perspectives. Explore fusion mechanisms based on attention transfer or causal association propagation. For instance, employ global-level feature importance rankings as prior knowledge to guide local counterfactual perturbations, or constrain local explanations through global causal structures to prevent the partiality inherent in isolated sample interpretations.

Challenge 5: Different application scenarios.

The scenarios covered in this paper face similar challenges in code vulnerability detection: insufficient domain high-quality datasets, supply chain risks (e.g., hardware backdoors or contaminated toolchains), and the requirement for deep analysis of business logic vulnerabilities

(e.g., financial contract rules). Each scenario also presents unique characteristics: mobile applications face constraints from device fragmentation and code obfuscation; smart contracts demand cross-contract tracking and financial knowledge integration; while LLM-based code generation is susceptible to training data contamination (Linux kernel cves 2025) and adversarial prompt attacks. Future research should focus on: 1) developing techniques for high-quality domain dataset collection and processing; 2) technology level, mobile applications need to combine dynamic sandboxing and large model causal reasoning, smart contracts should develop lightweight symbolic execution engines, code generation should integrate techniques (e.g., RAG, Agent, CoT) to correct model illusion; 3) establishing component review and defence mechanisms for supply chain security; and 4) implementing full-link protection systems, particularly for code generation scenarios that require guardrails spanning from training data purification to input/output filtering.

Conclusion

This study systematically searched the literature on deep learning-based source code vulnerability detection over the last four years, and using a multidimensional literature analysis framework, it not only covered an in-depth analysis of the dataset and model architecture evolution but also integrated a comparative analysis of multiple different application scenarios. In comparison to the existing literature of similar reviews, this study has the following advantages: First, the study is relatively new, ensuring the timeliness of the research conclusions; Second, the research dimensions are more diverse, which covers the depth of the technology and the breadth of the application. Ultimately, this review summarises the current challenges and outlines future research directions for deep learning-based source code vulnerability detection. The research acknowledges one limitation: our treatment of LLMs focuses narrowly on enhanced sequence models (particularly Transformer-based architectures), while generalized LLMs may exhibit non-sequential architectures beyond this scope.

Abbreviations

LLMs	Large language models
CNVD	China national vulnerability database
APT	Advanced persistent threat
CISA	Cybersecurity and infrastructure security agency
CVE	Common vulnerabilities and exposures
NVD	National vulnerability database
NIST	National Institute of Standards and Technology
SCAP	Security content automation protocol
ASTs	Abstract syntax trees
CFGs	Control flow graphs
SARD	Software assurance reference dataset
OWASP	Open web application security project

Juliet	Juliet 1.1 C/C++ and java test suite
PL	Programming language
NL	Natural language
MLM	Masked language modellings
VQs	Vulnerability queries
RAG	Retrieval augmented generation
CoT	Chain-of-thought
DFGs	Data flow graphs
GNNs	Graph neural networks
PDGs	Program dependency graphs
LSTM	Long short-term memory
GCNs	Graph convolutional networks
GATs	Graph attention networks
IRC	Intermediate representation of code
AI	Artificial intelligence
CW	Context window
AUC-ROC	Area under the receiver operating characteristic curve
FPR	False positive rate
FNR	False negative rate
TP	True positive
FP	False positive
TN	True negative
FN	False negative
CC	Code coverage
Acc	Accuracy
DoS	Denial of service
CSDN	Chinese software developer network
PII	Personally identifiable information

Acknowledgements

We would like to extend our heartfelt gratitude to the editor and the reviewers of this paper. Their professional advice and valuable feedback significantly contributed to the quality of this research.

Author contributions

All authors read and approved the manuscript.

Funding

This work was supported by the National Key Research and Development Program of China (No. 2023YFB3106302).

Availability of data and materials

Not applicable.

Declarations

Competing interests

The authors declare that they have no conflict of interests.

Received: 1 July 2025 Accepted: 3 November 2025

Published: 4 January 2026

References

- Ahmad WU, Chakraborty S, Ray B, Chang K (2021) Unified pre-training for program understanding and generation. In: Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, NAACL-HLT 2021, Online, June 6–11, 2021, pp. 2655–2668. <https://doi.org/10.18653/V1/2021.NAACL-MAIN.211>
- Anand A, Gupta A, Yadav N, Bajaj S (2024) A comprehensive survey of ai-driven advancements and techniques in automated program repair and code generation. CoRR abs/2411.07586 [arXiv:2411.07586](https://arxiv.org/abs/2411.07586)
- Basic E, Giarretta A (2024) Large language models and code security: A systematic literature review. CoRR abs/2412.15004 [arXiv:2412.15004](https://arxiv.org/abs/2412.15004)
- Bayoudh K (2024) A survey of multimodal hybrid deep learning for computer vision: Architectures, applications, trends, and challenges. Inf Fusion 105:102217. <https://doi.org/10.1016/J.INFFUS.2023.102217>
- Bhandari GP, Naseer A, Moonen L (2021) Cvefixes: automated collection of vulnerabilities and their fixes from open-source software. In: PROMISE '21: 17th International Conference on Predictive Models and Data Analytics in Software Engineering, Athens Greece, August 19–20, 2021, pp. 30–39. <https://doi.org/10.1145/3475960.3475985>
- Bhattacharjee A, Moraffah R, Garland J, Liu H (2024) Llms as counterfactual explanation modules: can chatgpt explain blackbox text classifiers. arXiv preprint [arXiv:2309.13340](https://arxiv.org/abs/2309.13340)
- Boland T, Black PE (2012) Juliet 1.1 c/c++ and java test suite. Computer 45(10):88–90
- Bresil M, Prasad PWC, Sayeed MS, Bakar UA (2025) Deep learning-based vulnerability detection solutions in smart contracts: A comparative and meta-analysis of existing approaches. IEEE Access 13:28894–28919. <https://doi.org/10.1109/ACCESS.2025.3532326>
- Brown TB, Mann B, Ryder N (2020) Language models are few-shot learners. In: Advances in Neural Information Processing Systems 2020, NeurIPS 2020, December 6–12, 2020, Virtual
- CISA (2025) <https://www.cisa.gov/resources-tools/services/network-reporting>. Accessed 30 March 2025
- CSDN (2025) <https://csdn.gitcode.host/Survey-Report-on-Developers-in-China/survey/2022/>. Accessed 30 March 2025
- Cao X, Wang J, Wu P, Fang Z (2024) Vulmpff: a vulnerability detection method for fusing code features in multiple perspectives. IET Inf Secur 2024:1–15. <https://doi.org/10.1049/2024/4313185>
- Cao D, Jun W (2024) Llm-cloudsec: Large language model empowered automatic and deep vulnerability analysis for intelligent clouds. In: IEEE INFOCOM 2024 - IEEE Conference on Computer Communications Workshops, Vancouver, BC, Canada, May 20, 2024, pp. 1–6. <https://doi.org/10.1109/INFOCOMWKSHP61880.2024.10620804>
- Cao S, Sun X, Wu X, Lo D, Bo L, Li B, Liu W (2024) Coca: Improving and explaining graph neural network-based vulnerability detection systems. In: Proceedings of the 46th IEEE/ACM International Conference on Software Engineering, ICSE 2024, Lisbon, Portugal, April 14–20, 2024, pp. 155–115513. <https://doi.org/10.1145/3597503.3639168>
- Chakraborty P, Arumugam KK, Alfadel M, Nagappan M, McIntosh S (2024) Revisiting the performance of deep learning-based vulnerability detection on realistic datasets. IEEE Trans Software Eng 50(8):2163–2177. <https://doi.org/10.1109/TSE.2024.3423712>
- Chakraborty S, Krishna R, Ding Y, Ray B (2022) Deep learning based vulnerability detection: Are we there yet? IEEE Trans Software Eng 48(9):3280–3296. <https://doi.org/10.1109/TSE.2021.3087402>
- Chang J, Ma Z, Cao B, Zhu E (2023) VDDA: an effective software vulnerability detection model based on deep learning and attention mechanism. In: 26th International Conference on Computer Supported Cooperative Work in Design, CSCWD 2023, Rio de Janeiro, Brazil, May 24–26, 2023, pp. 474–479. <https://doi.org/10.1109/CSCWD57460.2023.10152764>
- Che Z, Purushotham S, Khemani R, Liu Y (2015) Distilling knowledge from deep networks with applications to healthcare domain. Annales De Chirurgie
- Checkmarx (2025) <https://www.checkmarx.com/>. Accessed 30 March 2025
- Chen H, Covert IC, Lundberg SM, Lee S-I (2023) Algorithms to estimate shapley value feature attributions. Nature Machine Intelligence 5(6):590–601
- Chen Y, Ding Z, Alowain L, Chen X, Wagner DA (2023) Diversevul: A new vulnerable source code dataset for deep learning based vulnerability detection. In: Proceedings of the 26th International Symposium on Research in Attacks, Intrusions and Defenses, RAID 2023, Hong Kong, China, October 16–18, 2023, pp. 654–668. <https://doi.org/10.1145/3607199.3607242>
- Cheng X, Wang H, Hua J, Xu G, Sui Y (2021) Deepwukong: Statically detecting software vulnerabilities using deep graph neural network. ACM Trans Softw Eng Methodol 30(3):38–13833. <https://doi.org/10.1145/3436877>
- Cheng B, Wang K, Gao C, Luo X, Li L, Guo Y, Chen X, Wang H (2024) Slicelocator: Locating vulnerable statements with graph-based detectors
- Chenxi Y, Le W (2023) Cscvd: A method of source code vulnerability detection based on semantic feature and context dependence. In: 2023 9th International Conference on Computer and Communications (ICCC), pp. 2162–2169. IEEE
- Chess B, McGraw G (2004) Static analysis for security. IEEE Secur Priv 2(6):76–79. <https://doi.org/10.1109/MSP.2004.111>

- Chu H, Zhang P, Dong H, Xiao Y, Ji S, Li W (2023) A survey on smart contract vulnerabilities: data sources, detection and repair. *Inf Softw Technol* 159:107221. <https://doi.org/10.1016/J.INFSOF.2023.107221>
- Chu Z, Wan Y, Li Q, Wu Y, Zhang H, Sui Y, Xu G, Jin H (2024) Graph neural networks for vulnerability detection: A counterfactual explanation. In: Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis, ISSTA 2024, Vienna, Austria, September 16–20, 2024, pp. 389–401. <https://doi.org/10.1145/3650212.3652136>
- CodeXGLUE (2025) <https://samate.nist.gov/SRD/index.php>. Accessed 30 March 2025
- Croft R, Babar MA, Kholoosi MM (2023) Data quality for software vulnerability datasets. In: 45th IEEE/ACM International Conference on Software Engineering, ICSE 2023, Melbourne, Australia, May 14–20, 2023, pp. 121–133. <https://doi.org/10.1109/ICSE48619.2023.00022>
- DeepSeek-AI, Guo D, Yang D et al (2025) Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning. *CoRR abs/2501.12948* [arXiv:2501.12948](https://arxiv.org/abs/2501.12948)
- Deng X, Duan F, Xie R, Ye W, Zhang S (2024) Improving long-tail vulnerability detection through data augmentation based on large language models. In: IEEE International Conference on Software Maintenance and Evolution, ICSME 2024, Flagstaff, AZ, USA, October 6–11, 2024, pp. 262–274. <https://doi.org/10.1109/ICSME58944.2024.00033>
- Ding T (2024) Mobileagent: enhancing mobile control via human-machine interaction and SOP integration. *CoRR abs/2401.04124* [arXiv:2401.04124](https://arxiv.org/abs/2401.04124)
- Ding A, Li G, Yi X, Liu Y (2023) Ivsign: Interpretable vulnerability signature via code embedding and static analysis. In: 53rd Annual IEEE/IFIP International Conference on Dependable Systems and Networks, DSN 2023 - Workshops, Porto, Portugal, June 27–30, 2023, pp. 25–31. <https://doi.org/10.1109/DSN-W58399.2023.00025>
- Ding Y, Steenhoek B, Pei K, Kaiser GE, Le W, Ray B (2024) TRACED: execution-aware pre-training for source code. In: Proceedings of the 46th IEEE/ACM International Conference on Software Engineering, ICSE 2024, Lisbon, Portugal, April 14–20, 2024, pp. 36–13612. <https://doi.org/10.1145/3597503.3608140>
- Diwan A, Li MQ, Fung BCM (2022) Vdgraph2vec: Vulnerability detection in assembly code using message passing neural networks. In: 21st IEEE International Conference on Machine Learning and Applications, ICMLA 2022, Nassau, Bahamas, December 12–14, 2022, pp. 1039–1046. <https://doi.org/10.1109/ICMLA55696.2022.00173>
- Doll J, McCarthy C, McDougall H, Bhunia S (2025) Unraveling log4shell: Analyzing the impact and response to the log4j vulnerability. *CoRR abs/2501.17760* [arXiv:2501.17760](https://arxiv.org/abs/2501.17760)
- Dong L, Li W (2023) Reentrancy vulnerability detection based on semantic-aware graph convolutional neural network. In: 2023 3rd International Conference on Computer Science and Blockchain (CCSB), pp. 190–194. IEEE
- Dozono K, Gasiba TE, Stocco A (2024) Large language models for secure code assessment: A multi-language empirical study. *CoRR abs/2408.06428* [arXiv:2408.06428](https://arxiv.org/abs/2408.06428)
- Du G, Chen L, Wu T, Zheng X, Cui N, Shi G (2023) Cross domain on snippets: Bilstm-textcnn based vulnerability detection with domain adaptation. In: 26th International Conference on Computer Supported Cooperative Work in Design, CSCWD 2023, Rio de Janeiro, Brazil, May 24–26, 2023, pp. 1896–1901. <https://doi.org/10.1109/CSCWD57460.2023.10152772>
- Du G, Chen L, Wu T, Zheng X, Shi G (2024) Code property graph based cross-domain vulnerability detection via deep fused feature. In: IEEE International Symposium on Circuits and Systems, ISCAS 2024, Singapore, May 19–22, 2024, pp. 1–5. <https://doi.org/10.1109/ISCAS58744.2024.10558497>
- Du X, Zheng G, Wang K, Feng J, Deng W, Liu M, Chen B, Peng X, Ma T, Lou Y (2024) Vul-rag: Enhancing llm-based vulnerability detection via knowledge-level RAG. *CoRR abs/2406.11147* [arXiv:2406.11147](https://arxiv.org/abs/2406.11147)
- Espejel JL, Alassan MSY, Chouham EM, Dahhane W, Ettifouri EH (2023) A comprehensive review of state-of-the-art methods for java code generation from natural language text. *Nat Language Process J* 3:100013
- Fan J, Li Y, Wang S, Nguyen TN (2020) A C/C++ code vulnerability dataset with code changes and CVE summaries. In: MSR '20: 17th International Conference on Mining Software Repositories, Seoul, Republic of Korea, 29–30 June, 2020, pp. 508–512. <https://doi.org/10.1145/3379597.3387501>
- Feng Z, Guo D, Tang D, Duan N, Feng X, Gong M, Shou L, Qin B, Liu T, Jiang D, Zhou M (2020) Codebert: a pre-trained model for programming and natural languages. In: Findings of the Association for Computational Linguistics: EMNLP 2020, Online Event, 16–20 November 2020. Findings of ACL, vol. EMNLP 2020, pp. 1536–1547. <https://doi.org/10.18653/V1/2020.FINDINGS-EMNLP.139>
- Fu M, Nguyen V, Tantithamthavorn C, Phung D, Le T (2024) Vision transformer inspired automated vulnerability repair. *ACM Trans Softw Eng Methodol* 33(3):78–17829. <https://doi.org/10.1145/3632746>
- Fu Y, Liang P, Tahir A, Li Z, Shahin M, Yu J (2023) Security weaknesses of copilot generated code in github. *CoRR abs/2310.02059* [arXiv:2310.02059](https://arxiv.org/abs/2310.02059)
- Ganz T (2024) Software defect localization using explainable deep learning. PhD thesis, Technical University of Berlin, Germany. <https://nbn-resolving.org/urn:nbn:de:101:1-2405220157084.235098239515>
- Gao L, Guan L (2023) Interpretability of machine learning: recent advances and future prospects. *IEEE Multimed* 30(4):105–118. <https://doi.org/10.1109/MMUL.2023.3272513>
- Gao J, Li L, Kong P, Bissyandé TF, Klein J (2021) Understanding the evolution of android app vulnerabilities. *IEEE Trans Reliab* 70(1):212–230. <https://doi.org/10.1109/TR.2019.2956690>
- Garg S, Baliyan N (2021) Android security assessment: a review, taxonomy and research gap study. *Comput Secur* 100:102087
- Gujar SS (2024) Detectbert: Code vulnerability detection. In: 2024 Global Conference on Communications and Information Technologies (GCCIT), pp. 1–21. IEEE
- Guo D, Lu S, Duan N, Wang Y, Zhou M, Yin J (2022) Unixcoder: Unified cross-modal pre-training for code representation. In: Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), ACL 2022, Dublin, Ireland, May 22–27, 2022, pp. 7212–7225. <https://doi.org/10.18653/V1/2022.ACL-LONG.499>
- Guo D, Ren S, Lu S, Feng Z, Tang D, Liu S, Zhou L, Duan N, Svyatkovskiy A, Fu S, Tufano M, Deng SK, Clement CB, Drain D, Sundaresan N, Yin J, Jiang D, Zhou M (2021) Graphcodebert: Pre-training code representations with data flow. In: 9th International Conference on Learning Representations, ICLR 2021, Virtual Event, Austria, May 3–7, 2021
- Hassija V, Chamola V, Mahapatra A, Singal A, Goel D, Huang K, Scardapane S, Spinelli I, Mahmud M, Hussain A (2024) Interpreting black-box models: a review on explainable artificial intelligence. *Cogn Comput* 16(1):45–74. <https://doi.org/10.1007/S12559-023-10179-8>
- Hin D, Kan A, Chen H, Babar MA (2022) Linevd: Statement-level vulnerability detection using graph neural networks. In: 19th IEEE/ACM International Conference on Mining Software Repositories, MSR 2022, Pittsburgh, PA, USA, May 23–24, 2022, pp. 596–607. <https://doi.org/10.1145/3524842.3527949>
- Hosain MT, Jim JR, Mridha MF, Kabir MM (2024) Explainable AI approaches in deep learning: advancements, applications and challenges. *Comput Electr Eng* 117:109246. <https://doi.org/10.1016/J.COMPELECENG.2024.109246>
- Hovemeyer D, Pugh WW (2004) Finding bugs is easy. *ACM SIGPLAN Notices* 39(12):92–106. <https://doi.org/10.1145/1052883.1052895>
- Hu X, Xiong T, Yi B, Wei Z, Xiao R, Chen Y, Ye J, Tao M, Zhou X, Zhao Z (2025) OS agents: A survey on mllm-based agents for computer, phone and browser use. In: Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), ACL 2025, Vienna, Austria, July 27–August 1, 2025, pp. 7436–7465
- Huang Y, He M, Wang X, Zhang J (2024) Hevuld: a static vulnerability detection method using heterogeneous graph code representation. *IEEE Trans Inf Forensics Secur* 19:9129–9144. <https://doi.org/10.1109/TIFS.2024.3457162>
- Husain H, Wu H, Gazit T, Allamanis M, Brockschmidt M (2019) Codesearch-net challenge: Evaluating the state of semantic code search. *CoRR abs/1909.09436* [arXiv:1909.09436](https://arxiv.org/abs/1909.09436)
- Jesse K, Ahmed T, Devanbu PT, Morgan E (2023) Large language models and simple, stupid bugs. In: 20th IEEE/ACM International Conference on Mining Software Repositories, MSR 2023, Melbourne, Australia, May 15–16, 2023, pp. 563–575. <https://doi.org/10.1109/MSR59073.2023.00082>

- JianJie Y, Le W (2023) Code defect detection method based on bert and ensemble. In: 2023 9th International Conference on Computer and Communications (ICCC), pp. 2130–2138. IEEE
- Jiang J, Wang F, Shen J, Kim S, Kim S (2024) A survey on large language models for code generation. arXiv preprint [arXiv:2406.00515](https://arxiv.org/abs/2406.00515)
- Kaniewski S, Holstein D, Schmidt F, Heer T (2024) Vulnerability handling of ai-generated code - existing solutions and open challenges. CoRR abs/2408.08549 [arXiv:2408.08549](https://arxiv.org/abs/2408.08549)
- Ke-xin X, Tao L, Qin Y, Meng-qing Q (2023) PDGcross: source code vulnerability detection based on cross-file graph representation. Comput Technol Dev 33(8):102–107
- Kenesova AZ, Ilyasov A, Kim YR (2022) Programming language analysis for mobile app development
- Kharma M, Choi S, Alkhanafseh M, Mohaisen D (2025) Security and quality in LLM-generated code: A multi-language, multi-model analysis. CoRR abs/2502.01853 [arXiv:2502.01853](https://arxiv.org/abs/2502.01853)
- Kim S, Choi J, Ahmed ME, Nepal S, Kim H (2022) Vuldebert: A vulnerability detection system using BERT. In: IEEE International Symposium on Software Reliability Engineering Workshops, ISSRE 2022 - Workshops, Charlotte, NC, USA, October 31–Nov. 3, 2022, pp. 69–74. <https://doi.org/10.1109/ISSREW55968.2022.00042>
- Kraker W, Vranken H, Hommersom A (2023) GLICE: combining graph neural networks and program slicing to improve software vulnerability detection. In: IEEE European Symposium on Security and Privacy, EuroS & P 2023—Workshops, Delft, Netherlands, July 3–7, 2023, pp. 34–41. <https://doi.org/10.1109/EUROSPW59978.2023.00009>
- Kroeger N, Ley D, Krishna S, Agarwal C, Lakkaraju H (2023) Are large language models post hoc explainers?
- Kumar NK, Honnunar NV, Prakash MS, Lohith J (2024) Vulnerabilities in smart contracts: A detailed survey of detection and mitigation methodologies. In: 2024 International Conference on Emerging Technologies in Computer Science for Interdisciplinary Applications (ICETCS), pp. 1–7. IEEE
- Lakshmanarao A, Shashi M (2022) Android malware detection with deep learning using RNN from opcode sequences. Int J Interact Mob Technol 16(1) <https://doi.org/10.3991/IJIM.V16I01.26433>
- Le Y, Li H, Wang B, Luo Z, Yang A, Ma Z (2024) Supply chain security: pre-training model for python source code vulnerability detection. In: 2024 3rd International Joint Conference on Information and Communication Engineering (JCICE), pp. 150–155. IEEE
- Le T, Nguyen T, Le T, Phung DQ, Montague P, Vel OY, Qu L (2019) Maximal divergence sequential autoencoder for binary software vulnerability detection. In: 7th International Conference on Learning Representations, ICLR 2019, New Orleans, LA, USA, May 6–9, 2019
- Li X-H, Cao CC, Shi Y, Bai W, Gao H, Qiu L, Wang C, Gao Y, Zhang S, Xue X (2020) A survey of data-driven and knowledge-aware explainable AI. IEEE Trans Knowl Data Eng 34(1):29–49
- Li L, Ding SHH, Tian Y, Fung BCM, Charland P, Ou W, Song L, Chen C (2023) Vulanalyzer: explainable binary vulnerability detection with multi-task learning and attentional graph convolution. ACM Trans Priv Secur 26(3):28–12825. <https://doi.org/10.1145/3585386>
- Li Z, Zou D, Xu S, Chen Z, Zhu Y, Jin H (2022) Vuldeelocator: a deep learning-based fine-grained vulnerability detector. IEEE Trans Depend Secur Comput 19(4):2821–2837. <https://doi.org/10.1109/TDSC.2021.3076142>
- Li Y, Li X, Wu H, Zhang Y, Cheng X, Zhong S, Xu F (2024) Attention is all you need for llm-based code vulnerability localization. CoRR abs/2410.15288 [arXiv:2410.15288](https://arxiv.org/abs/2410.15288)
- Li Y, Wang S, Nguyen TN (2021) Vulnerability detection with fine-grained interpretations. In: ESEC/FSE '21: 29th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering, Athens, Greece, August 23–28, 2021, pp. 292–303. <https://doi.org/10.1145/3468264.3468597>
- Li Z, Zou D, Xu S, Ou X, Jin H, Wang S, Deng Z, Zhong Y (2018) Vuldeepecker: A deep learning-based system for vulnerability detection. In: 25th Annual Network and Distributed System Security Symposium, NDSS 2018, San Diego, California, USA, February 18–21, 2018
- Liang C, Wei Q, Jiang Z, Wang Y, Du J (2024) A source code vulnerability detection method based on adaptive graph neural networks. In: Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering Workshops, ASEW 2024, Sacramento, CA, USA, 27 October 2024–1 November 2024, pp. 187–196. <https://doi.org/10.1145/3691621.3694950>
- Lin Y, Li Y, Gu M, Sun H, Yue Q, Hu J, Cao C, Zhang Y (2022) Vulnerability dataset construction methods applied to vulnerability detection: A survey. In: 52nd Annual IEEE/IFIP International Conference on Dependable Systems and Networks, DSN Workshops 2022, Baltimore, MD, USA, June 27–30, 2022, pp. 141–146. <https://doi.org/10.1109/DSN-W54100.2022.00032>
- Lin J, Mohaisen D (2025) From large to mammoth: A comparative evaluation of large language models in vulnerability detection. In: 32nd Annual Network and Distributed System Security Symposium, NDSS 2025, San Diego, California, USA, February 24–28, 2025
- Linux kernel CVEs (2025) <https://www.linuxkernelcves.com/>. Accessed 30 March 2025
- Liu Y, Wang C, Ma Y (2024) DL4SC: a novel deep learning-based vulnerability detection framework for smart contracts. Autom Softw Eng 31(1):24. <https://doi.org/10.1007/S10515-024-00418-Z>
- Liu J, Xia CS, Wang Y, Zhang L (2023) Is your code generated by chatgpt really correct? rigorous evaluation of large language models for code generation. Adv Neural Inf Process Syst 36:21558–21572
- Liu S, Wu B, Xie X, Meng G, Liu Y (2023) Contrabert: Enhancing code pre-trained models via contrastive learning. In: 45th IEEE/ACM International Conference on Software Engineering, ICSE 2023, Melbourne, Australia, May 14–20, 2023, pp. 2476–2487. <https://doi.org/10.1109/ICSE48619.2023.00207>
- Liu Z, Zhong X, Deng X, Hong S, Gao X, Sun H (2024) Scalable defect detection via traversal on code graph. CoRR abs/2406.08098 [arXiv:2406.08098](https://arxiv.org/abs/2406.08098)
- Lundberg SM, Lee S (2017) A unified approach to interpreting model predictions. In: Advances in Neural Information Processing Systems 30: Annual Conference on Neural Information Processing Systems 2017, December 4–9, 2017, Long Beach, CA, USA, pp. 4765–4774
- Luo S, Ivson H, Han SC, Poon J (2024) Local interpretations for explainable natural language processing: a survey. ACM Comput Surv 56(9):232–123236. <https://doi.org/10.1145/3649450>
- Mahyari AA (2024) Harnessing the power of llms in source code vulnerability detection. In: IEEE Military Communications Conference, MILCOM 2024, Washington, DC, USA, October 28–Nov 1, 2024, pp. 251–256. <https://doi.org/10.1109/MILCOM61039.2024.10774025>
- Mao Q, Li Z, Hu X, Liu K, Xia X, Sun J (2024) Towards explainable vulnerability detection with large language models
- Mao Z, Li J, Jin D, Li M, Tei K (2024) Multi-role consensus through llms discussions for vulnerability detection. In: 24th IEEE International Conference on Software Quality, Reliability, and Security, QRS - Companion, Cambridge, United Kingdom, July 1–5, 2024, pp. 1318–1319. <https://doi.org/10.1109/QRS-C63300.2024.00173>
- Marchetto A (2024) Can explainability and deep-learning be used for localizing vulnerabilities in source code? In: Proceedings of the 5th ACM/IEEE International Conference on Automation of Software Test (AST 2024), pp. 110–119
- McCully GA, Hastings JD, Xu S, Fortier A (2024) Bi-directional transformers vs. word2vec: Discovering vulnerabilities in lifted compiled code. CoRR abs/2405.20611 [arXiv:2405.20611](https://arxiv.org/abs/2405.20611)
- Mishra DP, Senapati S, Dey T, Lenka RK (2024) Smart contract: Tools and challenges. In: 2024 1st International Conference on Cognitive, Green and Ubiquitous Computing (IC-CGU), pp. 1–6. IEEE
- Mohd Illzam MIE, Yong KSC, Then PHH, Yong SK (2023) Comparative analysis of graph representation techniques for code vulnerability detection. In: 2023 International Conference on Engineering and Emerging Technologies (ICEET), pp. 1–6. IEEE
- Mohsin A, Janicke H, Wood A, Sarker IH, Maglaras L, Janjua N (2024) Can we trust large language models generated code? A framework for in-context learning, security patterns, and code evaluations across diverse LLMs. arXiv preprint [arXiv:2406.12513](https://arxiv.org/abs/2406.12513)
- National vulnerability database (2025) <https://nvd.nist.gov/>. Accessed 30 March 2025
- Negri-Ribalta C, Geraud-Stewart R, Sergeeva A, Lenzini G (2024) A systematic literature review on the impact of AI models on the security of code generation. Front Big Data 7:1386720
- Neuhaus S, Zimmermann T, Holler C, Zeller A (2007) Predicting vulnerable software components. In: Proceedings of the 2007 ACM Conference on Computer and Communications Security, CCS 2007, Alexandria,

- Virginia, USA, October 28–31, 2007, pp. 529–540. <https://doi.org/10.1145/1315245.1315311>
- Nguyen HH, Viviani J-L, Ben Jabeur S (2023) Bankruptcy prediction using machine learning and shapley additive explanations. *Rev Quant Finance Acc*, 1–42
- Ni C, Guo X, Zhu Y, Xu X Yang X (2023) Function-level vulnerability detection through fusing multi-modal knowledge. In: 38th IEEE/ACM International Conference on Automated Software Engineering, ASE 2023, Luxembourg, September 11–15, 2023, pp. 1911–1918. <https://doi.org/10.1109/ASE56229.2023.00084>
- Ni C, Shen L, Yang X, Zhu Y, Wang S (2024) Megavul: A C/C++ vulnerability dataset with comprehensive code representations. In: 21st IEEE/ACM International Conference on Mining Software Repositories, MSR 2024, Lisbon, Portugal, April 15–16, 2024, pp. 738–742. <https://doi.org/10.1145/3643991.3644886>
- Nikitopoulos G, Dritsa K, Louridas P, Mitropoulos D (2021) Crossvul: a cross-language vulnerability dataset with commit data. In: ESEC/FSE '21: 29th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering, Athens, Greece, August 23–28, 2021, pp. 1565–1569. <https://doi.org/10.1145/3468264.3473122>
- Nong Y, Fang R, Yi G, Zhao K, Luo X, Chen F, Cai H (2024) VGX: large-scale sample generation for boosting learning-based software vulnerability analyses. In: Proceedings of the 46th IEEE/ACM International Conference on Software Engineering, ICSE 2024, Lisbon, Portugal, April 14–20, 2024, pp. 149–14913. <https://doi.org/10.1145/3597503.3639116>
- Owasp benchmark (2015) <https://owasp.org/www-project-benchmark/>. Accessed 30 June 2015
- Owasp LLM applications security (2025) <https://owasp.org/www-project-top-10-for-large-language-model-applications/>. Accessed 30 March 2025
- Owasp mobile security (2025) <http://www.owasp.org.cn/OWASP-CHINA/owasp-project/owasp-mobile-security/>. Accessed 30 March 2025
- Owasp smart contract security (2025) <https://owasp.org/www-project-smart-contract-top-10/>. Accessed 30 March 2025
- Parizi RM, Amritraj Dehghantanha A (2018) Smart contract programming languages on blockchains: An empirical evaluation of usability and security. In: Blockchain - ICBC 2018 - First International Conference, Held as Part of the Services Conference Federation, SCF 2018, Seattle, WA, USA, June 25–30, 2018, Proceedings. Lecture Notes in Computer Science, vol. 10974, pp. 75–91. https://doi.org/10.1007/978-3-319-94478-4_6
- Pedersen TL, Aut Benesty M (2017) lime: Local interpretable model-agnostic explanations
- Peng X, Wang S, Qin Y, Lin B, Chen L, Mao, X (2024) Keep it simple: Towards accurate vulnerability detection for large code graphs. *CoRR abs/2412.10164* [arXiv:2412.10164](https://arxiv.org/abs/2412.10164)
- Pope PE, Kolouri S, Rostami M, Martin CE, Hoffmann H (2019) Explainability methods for graph convolutional neural networks. In: IEEE Conference on Computer Vision and Pattern Recognition, CVPR 2019, Long Beach, CA, USA, June 16–20, 2019, pp. 10772–10781. <https://doi.org/10.1109/CVPR.2019.01103>
- Pornprasit C, Tantithamthavorn C, Thongtanunam P, Chen C (2023) D-ACT: towards diff-aware code transformation for code review under a time-wise evaluation. In: IEEE International Conference on Software Analysis, Evolution and Reengineering, SANER 2023, Taipa, Macao, March 21–24, 2023, pp. 296–307. <https://doi.org/10.1109/SANER56733.2023.00036>
- Purba MD, Ghosh A, Radford BJ, Chu B (2023) Software vulnerability detection using large language models. In: 34th IEEE International Symposium on Software Reliability Engineering, ISSRE 2023—Workshops, Florence, Italy, October 9–12, 2023, pp. 112–119. <https://doi.org/10.1109/ISSREW60843.2023.00058>
- Puri R, Kung DS, Janssen G, Zhang W, Domeniconi G, Zolotov V, Dolby J, Chen J, Choudhury MR, Decker L, Thost V, Buratti L, Pujar S, Ramji S, Finkler U, Malaika S, Reiss F (2021) Codenet: A large-scale AI for code dataset for learning a diversity of coding tasks. In: Proceedings of the Neural Information Processing Systems Track on Datasets and Benchmarks 1, NeurIPS Datasets and Benchmarks 2021, December 2021, Virtual
- Qi, W., Cao, J., Poddar, D., Li, S., Wang, X.: Enhancing pre-trained language models for vulnerability detection via semantic-preserving data augmentation. *CoRR abs/2410.00249* (2024) [arXiv:2410.00249](https://arxiv.org/abs/2410.00249)
- Rahman MM, Ceka I, Mao C, Chakraborty S, Ray B, Le W (2024). Towards Causal Deep Learning for Vulnerability Detection. <https://doi.org/10.1145/3597503.3639170>
- Russell RL, Kim LY, Hamilton LH, Lazovich T, Harer J, Ozdemir O, Ellingwood PM, McConley MW (2018) Automated vulnerability detection in source code using deep representation learning. In: 17th IEEE International Conference on Machine Learning and Applications, ICMLA 2018, Orlando, FL, USA, December 17–20, 2018, pp. 757–762. <https://doi.org/10.1109/ICMLA.2018.00120>
- Sarbakysh N, Wang Z (2023) A1BERT: A language-agnostic graph neural network model for vulnerability detection. In: 8th International Conference on Data Science in Cyberspace, DSC 2023, Hefei, China, August 18–20, 2023, pp. 205–212. <https://doi.org/10.1109/DSC59305.2023.00038>
- Sarkar A, Goyal A, Hicks D, Sarkar D, Hazra S (2019) Android application development: a brief overview of android platforms and evolution of security systems. In: 2019 Third International Conference on I-SMAC (IoT in Social, Mobile, Analytics and Cloud)(I-SMAC), pp. 73–79. IEEE
- Schnake T, Eberle O, Lederer J, Nakajima S, Schütt KT, Müller K, Montavon G (2022) Higher-order explanations of graph neural networks via relevant walks. *IEEE Trans Pattern Anal Mach Intell* 44(11):7581–7596. <https://doi.org/10.1109/TPAMI.2021.3115452>
- Senanayake J, Kalutarage HK, Al-Kadri MO, Petrovski A, Piras L (2023) Android source code vulnerability detection: a systematic literature review. *ACM Comput Surv* 55(9):187–118737. <https://doi.org/10.1145/3556974>
- Sharma A (2023) Android app development using kotlin programming language. In: AIP Conference Proceedings, vol. 2427. AIP Publishing
- Sheng Z, Wu F, Zuo X, Li C, Qiao Y, Hang L (2024) Lprotector: An llm-driven vulnerability detection system. *CoRR abs/2411.06493* [arXiv:2411.06493](https://arxiv.org/abs/2411.06493)
- Shi-Kun DXWR (2023) Survey of source code bug detection based on deep learning. *J Softw* 34(02):625–654
- Shihab A, Alanezi M (2024) Exploring datasets in software vulnerability analysis: A review. In: 2024 4th International Conference on Emerging Smart Technologies and Applications (eSmarTA), pp. 1–6. IEEE
- Shijian ZKFFG (2023) Source code vulnerability detection based on hybrid code representation. *J Comput Appl* 43(8), 2517–2526
- Shin Y, Williams LA (2008) An empirical model to predict security vulnerabilities using code complexity metrics. In: Proceedings of the Second International Symposium on Empirical Software Engineering and Measurement, ESEM 2008, October 9–10, 2008, Kaiserslautern, Germany, pp. 315–317. <https://doi.org/10.1145/1414004.1414065>
- Shouliang W (2023) A source code vulnerability detection scheme based on code sequence and graph structure. *J North Univ China (Natural Science Edition)* 44(6), 641–653
- Sotgiu A, Pintor M, Biggio B (2022) Explainability-based debugging of machine learning for vulnerability discovery. In: ARES 2022: The 17th International Conference on Availability, Reliability and Security, Vienna, Austria, August 23–26, 2022, pp. 113–11138. <https://doi.org/10.1145/3538969.3543809>
- Souza B, Pradel M (2023) Lexecutor: Learning-guided execution. In: Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering, ESEC/FSE 2023, San Francisco, CA, USA, December 3–9, 2023, pp. 1522–1534. <https://doi.org/10.1145/3611643.3616254>
- Su X, Zheng W, Jiang Y, Wei H, Wan J, Wei Z (2024) Research and progress on learning-based source code vulnerability detection. *Chin J Comput* 47(2):337–374
- Sultan MF, Karim T, Shaon MSH, Wardat M, Akter MS (2024) Enhanced llm-based framework for predicting null pointer dereference in source code. *CoRR abs/2412.00216* [arXiv:2412.00216](https://arxiv.org/abs/2412.00216)
- Sun H, Liu Y, Ding Z, Xiao Y, Hao Z, Zhu H (2023) An enhanced vulnerability detection in software using a heterogeneous encoding ensemble. In: IEEE Symposium on Computers and Communications, ISCC 2023, Gammarth, Tunisia, July 9–12, 2023, pp. 1214–1220. <https://doi.org/10.1109/ISCC58397.2023.10217978>
- Tan S, Caruana R, Hooker G, Lou Y (2018) Distill-and-compare: Auditing black-box models using transparent model distillation. In: Proceedings of the 2018 AAAI/ACM Conference on AI, Ethics, and Society, AIES 2018, New Orleans, LA, USA, February 02–03, 2018, pp. 303–310. <https://doi.org/10.1145/3278721.3278725>
- Tang X, Du Y, Lai A, Zhang Z, Shi L (2023) Deep learning-based solution for smart contract vulnerabilities detection. *Sci Rep* 13(1):20106
- Tang W, Tang M, Ban M, Zhao Z, Feng M (2023) CSGVD: a deep learning approach combining sequence and graph embedding for source code

- vulnerability detection. *J Syst Softw* 199:111623. <https://doi.org/10.1016/J.JSS.2023.111623>
- Thangavel M, TGR AS, Priyadharshini P, Saranya T (2022) Review on machine and deep learning applications for cyber security. In: *Research Anthology on Machine Learning Techniques, Methods, and Applications*, pp. 1143–1164
- Thapa C, Jang SI, Ahmed ME, Camtepe S, Pieprzyk J, Nepal S (2022) Transformer-based language models for software vulnerability detection. In: *Annual Computer Security Applications Conference, ACSAC 2022*, Austin, TX, USA, December 5–9, 2022, pp. 481–496. <https://doi.org/10.1145/3564625.3567985>
- Tihanyi N, Bisztray T, Ferrag MA, Jain R, Cordeiro LC (2025) How secure is AI-generated code: a large-scale comparison of large language models. *Empir Softw Eng* 30(2):47. <https://doi.org/10.1007/S10664-024-10590-1>
- Tihanyi N, Bisztray T, Jain R, Ferrag MA, Cordeiro LC, Mavroedis V (2023) The formal dataset: Generative AI in software security through the lens of formal verification. In: *Proceedings of the 19th International Conference on Predictive Models and Data Analytics in Software Engineering, PROMISE 2023*, San Francisco, CA, USA, 8 December 2023, pp. 33–43. <https://doi.org/10.1145/3617555.3617874>
- Touvron H, Lavril T, Izacard G, Martinet X, Lachaux M, Lacroix T, Rozière B, Goyal N, Hambro E, Azhar F, Rodriguez A, Joulin A, Grave E, Lample G (2023) Llama: Open and efficient foundation language models. *CoRR abs/2302.13971* [arXiv:2302.13971](https://arxiv.org/abs/2302.13971)
- Turpin M, Michael J, Perez E, Bowman SR (2023) Language models don't always say what they think: Unfaithful explanations in chain-of-thought prompting. In: *Advances in Neural Information Processing Systems 36: Annual Conference on Neural Information Processing Systems 2023, NeurIPS 2023*, New Orleans, LA, USA, December 10–16, 2023
- Vadisetty R, Polamarasetti A, Prajapati S, Butani JB, et al (2023) Leveraging generative ai for automated code generation and security compliance in cloud-based devops pipelines: A review. Available at SSRN 5218298
- Vaswani A, Shazeer N, Parmar N, Uszkoreit J, Jones L, Gomez AN, Kaiser L, Polosukhin I (2017) Attention is all you need. In: *Advances in Neural Information Processing Systems 30: Annual Conference on Neural Information Processing Systems 2017*, December 4–9, 2017, Long Beach, CA, USA, pp. 5998–6008
- Velickovic P, Cucurull G, Casanova A, Romero A, Liò P, Bengio Y (2017) Graph attention networks. *CoRR abs/1710.10903* [arXiv:1710.10903](https://arxiv.org/abs/1710.10903)
- Votipka D, Stevens R, Redmiles EM, Hu J, Mazurek ML (2018) Hackers vs. testers: A comparison of software vulnerability discovery processes. In: *2018 IEEE Symposium on Security and Privacy, SP 2018, Proceedings*, 21–23 May 2018, San Francisco, California, USA, pp. 374–391. <https://doi.org/10.1109/SP.2018.00003>
- Walia S, Kumar K, Agarwal S, Kim H (2022) Using XAI for deep learning-based image manipulation detection with shapley additive explanation. *Symmetry* 14(8):1611. <https://doi.org/10.3390/SYM14081611>
- Wang X, Hu R, Gao C, Wen X, Chen Y, Liao Q (2024) Reposvul: A repository-level high-quality vulnerability dataset. In: *Proceedings of the 2024 IEEE/ACM 46th International Conference on Software Engineering: Companion Proceedings, ICSE Companion 2024*, Lisbon, Portugal, April 14–20, 2024, pp. 472–483. <https://doi.org/10.1145/3639478.3647634>
- Wang Z, Li G, Li J, Xiong Y, Yan M, Jin Z (2024) M2cvd: Enhancing vulnerability semantic through multi-model collaboration for code vulnerability detection. *arXiv preprint* [arXiv:2406.05940](https://arxiv.org/abs/2406.05940)
- Wang J, Ni T, Lee W, Zhao Q (2025) A contemporary survey of large language model assisted program analysis. *CoRR abs/2502.18474* [arXiv:2502.18474](https://arxiv.org/abs/2502.18474)
- Wang Y, Wang W, Joty SR, Hoi SCH (2021) Codet5: Identifier-aware unified pre-trained encoder-decoder models for code understanding and generation. In: *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing, EMNLP 2021, Virtual Event / Punta Cana, Dominican Republic, 7–11 November, 2021*, pp. 8696–8708. <https://doi.org/10.18653/V1/2021.EMNLP-MAIN.685>
- Wang Z, Xiong L, Wang J, Li D (2024) Android malware detection based on multi-feature fusion and deep learning. In: *Fourth International Conference on Sensors and Information Technology (ICSI 2024)*, vol. 13107, pp. 585–595. SPIE
- Wang X, Zhu M, Bo D, Cui P, Shi C, Pei J (2020) AM-GCN: adaptive multi-channel graph convolutional networks. In: *KDD '20: The 26th ACM SIGKDD Conference on Knowledge Discovery and Data Mining, Virtual Event*, CA, USA, August 23–27, 2020, pp. 1243–1253. <https://doi.org/10.1145/3394486.3403177>
- Watson, A., Ufuktepe, E., Palaniappan, K.: Detecting software code vulnerabilities using 2d convolutional neural networks with program slicing feature maps. In: *51st IEEE Applied Imagery Pattern Recognition Workshop, AIPR 2022*, Washington, DC, USA, October 11–13, 2022, pp. 1–9 (2022). <https://doi.org/10.1109/AIPR57179.2022.10092211>
- Wen X, Gao C, Ye J, Li Y, Tian Z, Jia Y, Wang X (2024) Meta-path based attentional graph learning model for vulnerability detection. *IEEE Trans Softw Eng* 50(3):360–375. <https://doi.org/10.1109/TSE.2023.3340267>
- Wen X, Chen Y, Gao C, Zhang H, Zhang JM, Liao Q (2023) Vulnerability detection with graph simplification and enhanced graph representation learning. In: *45th IEEE/ACM International Conference on Software Engineering, ICSE 2023*, Melbourne, Australia, May 14–20, 2023, pp. 2275–2286. <https://doi.org/10.1109/ICSE48619.2023.00191>
- Wu C, Xiong J, Xiong H, Zhao Y, Yi W (2022) A review on recent progress of smart contract in blockchain. *IEEE Access* 10:50839–50863. <https://doi.org/10.1109/ACCESS.2022.3174052>
- Wu B, Zou F (2022) Code vulnerability detection based on deep sequence and graph models: a survey. *Secur Commun Netw* 2022(1):1176898
- Wu Z, Pan S, Chen F, Long G, Zhang C, Yu PS (2019) A comprehensive survey on graph neural networks. *CoRR abs/1901.00596* [arXiv:1901.00596](https://arxiv.org/abs/1901.00596)
- Wu H, Peng Y, He Y, Fan J (2024) A review of deep learning-based vulnerability detection tools for ethernet smart contracts. *CMES-Computer Modeling in Engineering & Sciences* **140**(1)
- Wu L, Wang C, Liu T, Zhao Y, Wang H (2025) From assistants to adversaries: Exploring the security risks of mobile LLM agents. *arXiv preprint* [arXiv:2505.12981](https://arxiv.org/abs/2505.12981)
- Wu Y, Zou D, Dou S, Yang W, Xu D, Jin H (2022) Vulcnn: An image-inspired scalable vulnerability detection system. In: *44th IEEE/ACM 44th International Conference on Software Engineering, ICSE 2022*, Pittsburgh, PA, USA, May 25–27, 2022, pp. 2365–2376. <https://doi.org/10.1145/3510003.3510229>
- Xiao P, Xiao Q, Zhang X, Wu Y, Yang F (2024) Vulnerability detection based on enhanced graph representation learning. *IEEE Trans Inf Forensics Secur* 19:5120–5135. <https://doi.org/10.1109/TIFS.2024.3392536>
- Xing, Y., Huang, J., Wang, G., You, Y., Yang, B., Li, X., Luo, Y.: Data augmentation for vulnerability detection based on code refactoring and mixup. In: *35th IEEE International Symposium on Software Reliability Engineering, ISSRE 2024—Workshops*, Tsukuba, Japan, October 28–31, 2024, pp. 161–168 (2024). <https://doi.org/10.1109/ISSREW63542.2024.00070>
- Xu K, Park DH, Yi C, Sutton C (2018) Interpreting deep classifier by visual distillation of dark knowledge. *CoRR abs/1803.04042* [arXiv:1803.04042](https://arxiv.org/abs/1803.04042)
- Xue Z, Gao Z, Wang S, Hu X, Xia X, Li S (2024) Selfpico: Self-guided partial code execution with llms. In: *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis, ISSTA 2024*, Vienna, Austria, September 16–20, 2024, pp. 1389–1401. <https://doi.org/10.1145/3650212.3680368>
- Yamaguchi F, Lottmann M, Rieck K (2012) Generalized vulnerability extrapolation using abstract syntax trees. In: *28th Annual Computer Security Applications Conference, ACSAC 2012*, Orlando, FL, USA, 3–7 December 2012, pp. 359–368. <https://doi.org/10.1145/2420950.2421003>
- Yang J, Fu C, Deng F, Wen M, Guo X, Wan C (2023) Toward interpretable graph tensor convolution neural network for code semantics embedding. *ACM Trans Softw Eng Methodol* 32(5):115–111540. <https://doi.org/10.1145/3582574>
- Yang AZH, Tian H, Ye H, Martins R, Le Goues C (2024) Security vulnerability detection with multitask self-instructed fine-tuning of large language models. *CoRR abs/2406.05892* [arXiv:2406.05892](https://arxiv.org/abs/2406.05892)
- Yang G, Zhou Y, Chen X, Zhang X, Zhuo TY, Lo D, Chen T (2024) Dece: Deceptive cross-entropy loss designed for defending backdoor attacks. *arXiv preprint* [arXiv:2407.08956](https://arxiv.org/abs/2407.08956)
- Ying, Z., Bourgeois, D., You, J., Zitnik, M., Leskovec, J.: Gnnexplainer: Generating explanations for graph neural networks. In: *Advances in Neural Information Processing Systems 32: Annual Conference on Neural Information Processing Systems 2019, NeurIPS 2019*, December 8–14, 2019, Vancouver, BC, Canada, pp. 9240–9251 (2019)
- Yong C, Defeng H, Chao X, Nannan C, Jianbo L (2025) Smart contract generation model based on code annotation and AST-LSTM tuning. *J Supercomput* 81(5):731

- Yu KW, Lee BM (2025) Detecting rug-pull: analyzing smart contract backdoor codes in ethereum. *Appl Sci* 15(1):450
- Yu Z, Ananiadou S (2024) Interpreting arithmetic mechanism in large language models through comparative neuron analysis. In: Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing, EMNLP 2024, Miami, FL, USA, November 12–16, 2024, pp. 3293–3306
- Yuan Y, Li Z, Zhao B (2025) A survey of multimodal learning: methods, applications, and future. *ACM Comput Surv* 57(7):167–116734. <https://doi.org/10.1145/3713070>
- Yuan X, Lin G, Mei H, Tai Y, Zhang J (2024) Software vulnerable functions discovery based on code composite feature. *J Inf Secur Appl* 81:103718. <https://doi.org/10.1016/J.JISA.2024.103718>
- Yuan B, Lu Y, Fang Y, Wu Y, Zou D, Li Z, Li Z, Jin H (2023) Enhancing deep learning-based vulnerability detection by building behavior graph model. In: 45th IEEE/ACM International Conference on Software Engineering, ICSE 2023, Melbourne, Australia, May 14–20, 2023, pp. 2262–2274. <https://doi.org/10.1109/ICSE48619.2023.00190>
- Zhang H, Babar MA, Tell P (2011) Identifying relevant studies in software engineering. *Inf Softw Technol* 53(6):625–637. <https://doi.org/10.1016/J.INFSOF.2010.12.010>
- Zhang L, Wang J, Wang W, Jin Z, Su Y, Chen H (2022) Smart contract vulnerability detection combined with multi-objective detection. *Comput Netw* 217:109289. <https://doi.org/10.1016/J.COMNET.2022.109289>
- Zhang J (2024) Combining gpt and code-based similarity checking for effective smart contract vulnerability detection. *arXiv preprint arXiv:2412.18225*
- Zhang K, Li J, Li G, Shi X, Jin Z (2024) Codeagent: Enhancing code generation with tool-integrated agent systems for real-world repo-level coding challenges. *arXiv preprint arXiv:2401.07339*
- Zhang K, Zhang H, Li G, Li J, Li Z, Jin Z Toolcoder: Teach code generation models to use API search tools. *arXiv preprint arXiv:2305.04032*
- Zhen X, Hongjia L, Dan W (2023) Mobile Terminal Security Architecture and Key Technologies
- Zheng Y, Pujar S, Lewis BL, Buratti L, Epstein EA, Yang B, Laredo J, Morari A, Su Z (2021) D2A: A dataset built for ai-based vulnerability detection methods using differential analysis. In: 43rd IEEE/ACM International Conference on Software Engineering: Software Engineering in Practice, ICSE (SEIP) 2021, Madrid, Spain, May 25–28, 2021, pp. 111–120. <https://doi.org/10.1109/ICSE-SEIP52600.2021.00020>
- Zhong Q, Peng T, Chen S (2023) Vuldecrg: A vulnerability detection system based on code representation graph. In: 2023 2nd International Conference on Artificial Intelligence, Human-Computer Interaction and Robotics (AIHCIR), pp. 243–247. IEEE
- Zhou X, Chen Y, Guo H, Chen X, Huang Y (2023) Security code recommendations for smart contract. In: IEEE International Conference on Software Analysis, Evolution and Reengineering, SANER 2023, Taipa, Macao, March 21–24, 2023, pp. 190–200. <https://doi.org/10.1109/SANER56733.2023.00027>
- Zhou X, Han D, Lo D (2021) Assessing generalizability of codebert. In: IEEE International Conference on Software Maintenance and Evolution, ICSME 2021, Luxembourg, September 27–October 1, 2021, pp. 425–436. <https://doi.org/10.1109/ICSME52107.2021.00044>
- Zhou K, Hwang JD, Ren X, Sap M (2024) Relying on the unreliable: The impact of language models' reluctance to express uncertainty. *arXiv preprint arXiv:2401.06730*
- Zhou Y, Liu S, Siow JK, Du X, Liu Y (2019) Devign: Effective vulnerability identification by learning comprehensive program semantics via graph neural networks. In: Advances in Neural Information Processing Systems 32: Annual Conference on Neural Information Processing Systems 2019, NeurIPS 2019, December 8–14, 2019, Vancouver, BC, Canada, pp. 10197–10207
- Zhou X, Verma RM (2022) Software vulnerability detection via multimodal deep learning. In: Security and Trust Management - 18th International Workshop, STM 2022, Copenhagen, Denmark, September 29, 2022, Proceedings. Lecture Notes in Computer Science, vol. 13867, pp. 85–103. https://doi.org/10.1007/978-3-031-29504-1_5
- Zibaeirad A, Vieira M (2024) Vulnllmeval: A framework for evaluating large language models in software vulnerability detection and patching. *CoRR abs/2409.10756 arXiv:2409.10756*
- Zou H, Li Z, Li X (2025) Malicious code detection in smart contracts via opcode vectorization. *CoRR abs/2504.12720 arXiv:2504.12720*
- Zou D, Wang S, Xu S, Li Z, Jin H (2019) μ vuldeepecker: A deep learning-based system for multiclass vulnerability detection. *IEEE Trans Depend Secure Comput* PP(99), 1–1

Publisher's Note

Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Yan Zhang is currently an associate researcher at the Institute of Information Engineering, Chinese Academy of Sciences. Research interests include software security, system security, smart device security, etc.